Crest

Release 5.9.2

Wave Harmonic

May 13, 2026

TABLE OF CONTENTS

Part I

About

CHAPTER

ONE

INTRODUCTION

Crest is a technically advanced water system for Unity.

Designed for performance, it utilizes Level Of Detail (LOD) strategies and GPU acceleration to ensure fast updates and rendering. Crest is also highly flexible, allowing for custom inputs to LODs (such as shape or foam) and featuring an intuitive shape authoring interface.

This documentation is for Crest 5.9.2.

You can view the living documentation online.

1.1 Assets

Crest 5, and several assets which extend it, are available on the Unity Asset Store:

- Crest 5: Asset Store
- CPU Queries: Asset Store
- Paint: Asset Store
- Portals: Asset Store
- Shallow Water: Asset Store
- Shifting Origin: Asset Store
- Splines: Asset Store
- Whirlpool: Asset Store

You can also find *Crest 4* on the *Unity Asset Store* for all render pipelines.

- Crest Water 4 BIRP: Asset Store
- Crest Water 4 HDRP: Asset Store
- Crest Water 4 URP: Asset Store

1.2 Social

Important

If the Discord invite does not work when visiting one of the invite links, try pasting **JJjx9qcq83** into *Discord* → *Add a Server* → *Join a Server* in Discord.

- **Website** https://waveharmonic.com
- **Asset Store** https://assetstore.unity.com/publishers/41652
- **YouTube** https://www.youtube.com/@WaveHarmonic
- **Discord** https://discord.gg/JJjx9qcq83
- **X** https://x.com/WaveHarmonicX

CHAPTER

TWO

INTEGRATIONS

The following is a list of assets that have known integrations for Crest. This list is not exhaustive, and there may be assets that work without declaring support.

2.1 Atmosphere

Assets that handle atmosphere (ie sky and weather) often need integrating to apply their fog to transparent objects. Crest being a Shader Graph means any instructions that work for transparent Shader Graph should also work for Crest. The following have provided instructions to integrate with Crest or support Shader Graph generally.

- Cozy 3: Asset Store
- Enviro 3: Asset Store
- Expanse: Asset Store

For vendors, we recommend considering our *instructions* for weather asset integrations to give users the best experience.

2.2 Physics

Crest provides an API for querying the water which can be used for physics like buoyancy and drag.

- Dynamic Water Physics 2: Asset Store

2.3 World Building

World building assets can provide presets to place Crest into the world.

- Gaia Pro: Asset Store

CHAPTER

THREE

KNOWN ISSUES

We keep track of issues on Discord and GitHub.

Most of the issues on GitHub are for Crest 4. Please see the following links for Github:

- **Issues on GitHub.**
  - BIRP (Built-in Render Pipeline) specific issues on GitHub.
  - HDRP (High Definition Render Pipeline) specific issues on GitHub.
  - URP (Universal Render Pipeline) specific issues on GitHub.

If you discover a bug, please open a bug report or mention it on the bugs channel on our Discord.

Or join our Discord and see #bugs and #help forums.

Important

If the Discord invite does not work when visiting one of the invite links, try pasting **JJjx9qcq83** into *Discord* → *Add a Server* → *Join a Server* in Discord.

3.1 Unity Bugs

There are some Unity issues that affect Crest. Some of these may even be blocking new features from being developed. A list of these issues can be found on our wiki which can be voted on for Unity to prioritize.

As a reminder it is important to always use the latest patched Unity version.

3.2 Prefab Mode Not Supported

Crest does not support running in prefab mode which means dirty state in prefab mode will not be reflected in the scene view. Save the prefab to see the changes.

3.3 Missing Files

By default Unity Version Control ignores one of our directories. *ignore.conf* needs to be updated to not ignore all Library folders, only the one in the project root.

CHAPTER

FOUR

RELEASE NOTES

5.9.2

Fixes

- Fix waves sampled using geometry inputs
- Fix waterfall sample

5.9.1

Fixes

- Fix water missing if using a real-time Depth Probe which refreshes every frame
- Fix culling issues if *Time Slice Bounds Update Frame Count* is used with *Multiple Viewpoints* or *Editor Multiple Viewpoints* by not applying it, as it is not compatible

Optimizations

- Skip redundant render pipeline check in standalone

5.9.0

For this release the focus was on improving the documentation and auditing the codebase for reliability issues. That being said, a noteworthy improvement is that the Water Body (now Local Override) has received an expansion in capabilities to help with authoring scenes.

Changes

- Add bicubic sampling option to all wave inputs
- Add *Feather Directional* toggle to "Shape Waves/Add From Geometry" shader
- Add Randomize toggle to Shape Gerstner to disable randomization if amplitudes and phases
- Floating Object will not receive physics inside a Local Override which excludes physics
- Add HasWater API which tests whether inside Local Override or not
- Improve handling of no scene reload
- Add Storm sample

Local Override

- Rename Water Body to Local Override (UI only)
- Local Override can optionally add or remove water
- Local Override can now remove underwater and physics
- Local Override can take the Y axis into account for volume and physics
- Add dedicated foldout "Overrides" to Water Renderer for global local override settings
- Replace Water Body Culling with Default Excludes on the Water Renderer
- Default Excludes always applies even with no local overrides present
- Add toggle for Material Overrides
- Material Overrides can revert to the global materials if none provided
- Add Conservative option for removing chunks for the performance use case
- Smaller Local Overrides have higher priority
- Show global settings in Local Override inspector
- Rename Clip to Precision
- Rename Local Override *Material* to the more descriptive *Above Surface Material*
- Add Overrides heading to Local Override UI

Fixes

- Fix RT memory leak in Unity 6.4 when viewing the Water Renderer inspector
- Fix Unity 6.6 compilation errors
- Fix null exceptions in RT previews for Unity 6.4
- Fix reflection path running for Unity 6+ when not needed
- Fix obsolete API usage
- Fix borrowed arrays not being cleared before returning them to the pool which could prevent GC (CoreCLR)
- Fix listed not being cleared which could prevent GC (CoreCLR)
- Fix not guarding against division by zero wind speed in ShapeGerstner
- Fix ShapeGerstner destroying shared spectrum if there are multiple
- Fix copy-paste/duplication not destroying all hidden objects
- Fix queries potentially not being cleaned up
- Fix per-frame allocation
- Fix standalone crash on quit for Metal devices
- Fix several cases where managed objects were not being correctly disposed on deactivation
- Fix crash in editor on recompile if using planar reflections
- Fix several cases of memory leaks and exceptions on render pipeline change
- Fix several cases of memory leaks and exceptions on recompile
- Fix attenuation missing from physically based sky *HDRP*

- Fix possible jitters in LOD center
- Fix lifecycle events running twice on component creation in editor
- Fix potential destroying RT set as Camera.targetTexture
- Fix potential errors/issues on undo/redo
- Fix a case of shader error "X4567: maximum cbuffer exceeded"
- Fix some cases of light baking not working or double baking in sample scenes

Optimizations

- Optimize chunk culling by using forceRenderingOff instead of enabled
- Improve chunk culling accuracy (too conservative)
- Do not execute TIR camera if underwater is not active
- Cull surface and underwater passes if no visible chunks
- Cull remaining rendering pipeline if no surface or underwater active after chunk culling
- Minor Floating Object optimization
- Use non allocating GetComponentsInChildren
- Prevent BIRP shadow code path from running for SRPs
- Optimize per-frame checks
- Cull planar reflection above/below camera passes
- Reduce C# reflection usage
- Optimize render pipeline checks

Documentation

- Update website design: wider with banner and grid indices
- Add new parts Simulation and Appearance
- Split up many large pages into smaller ones
- Reorganize many pages
- Add Stylization, Reflection, Bioluminescence pages
- Expand on many areas
- Update links to Unity documentation to 6.3
- Fix typos

Full version history has been omitted for brevity. It can be found at Release Notes.

Part II

Getting Started

CHAPTER

FIVE

REQUIREMENTS

- Unity 2022.3 or later, latest patched version
- *Shader Graph* package
- Shader compilation target 4.5 or above
- Graphics API that is **not** OpenGL or WebGL

Important

It may be necessary to update to the latest patched version of Unity in order for a problem to be solved. Furthermore, legacy versions of Unity are not supported.

Sample scenes when using BIRP uses the post-processing package. If this is not present in your project, you will see an unassigned script warning which you can fix by removing the offending script. Furthermore, scenes will look overexposed.

5.1 Render Pipeline Setup

Ensure that your chosen render pipeline is setup and functioning, either by setting up a new project using the appropriate template or by configuring your current project. This is beyond the scope of this documentation so please see the Unity documentation (BIRP, HDRP, URP) for more information.

Switch to Linear space rendering under *Edit* → *Project Settings* → *Player* → *Other Settings*. If your platform(s) require Gamma space, the material settings will need to be adjusted to compensate. Please see the Unity documentation for more information.

Tip

If you are starting from scratch we recommend creating a project using a template in the Unity Hub.

CHAPTER

SIX

INSTALLATION

The steps to set up Crest in a new or existing project are as follows:

6.1 Importing Crest

Import the Crest package into the project using the Package Manager window in the Unity Editor.

Crest is a UPM package and thus is imported into the Packages directory. Once imported Crest packages will be listed in the package manager window. Familiarity with the package details pane is important as this is where Samples are imported from.

6.2 Troubleshooting

The following are issues with the install process which come up frequently.

I am seeing errors in the console and/or visual issues

When changing Unity versions, setting up a render pipeline or making changes to packages, the project can appear to break. This may manifest as spurious errors in the log, no water rendering, magenta materials, scripts unassigned in example scenes, etcetera. Often, restarting the Editor fixes it. Additionally, re-importing broken (ie magenta) materials/shaders may be required. Clearing out the *Library* folder can also help to reset the project and clear temporary errors. These issues are not specific to Crest, but we note them anyway as we find our users regularly encounter them.

I am seeing magenta materials after changing render pipelines

Unity has a bug where sometimes it will not correctly switch materials to the current render pipeline. Solutions can vary:

- Restarting Unity
- View the affected material's inspector
- Reimport the affected material/shader

This affects Shader Graph specifically. Find our water Shader Graph, right click and re-import it. It may appear as a blank file icon instead of the usual Shader Graph icon.

I can enter play mode, but errors appear in the log at runtime that mention missing 'kernels'

Recent versions of Unity have a bug that makes shader import unreliable. Please try reimporting the *Packages/Crest/Runtime/Shaders* folder using the right click menu in the project view. Or simply close Unity, delete the Library folder and restart which will trigger everything to reimport.

Why aren't my prefab mode edits not reflected in the scene view?

Crest does not support running in prefab mode which means dirty state in prefab mode will not be reflected in the scene view. Save the prefab to see the changes.

I am seeing "Crest does not support OpenGL/WebGL backends." in the editor

It is likely Unity has defaulted to using OpenGL on your platform. You will need to switch to a supported graphics API like Vulkan. You will need to make Vulkan the default by overriding the graphics APIs.

Why I am seeing "The referenced script on this Behavior is missing!" or similar in Crest's sample scenes and prefabs?

This is normal and can be ignored. The sample scenes support all render pipelines which require render pipeline specific components to be serialized in the scene. If a render pipeline package is missing, then those components will also be missing.

CHAPTER

SEVEN

MIGRATION

As a general migration requirement, if you have generated an HLSL shader from the Water Shader Graph, then you will need to regenerate if it changes.

7.1 Migrating from 5.6.6

The *Crest/Water* shader has renamed the flow keyword. Materials will automatically be upgraded in the background, but it can be run manually with *Edit* → *Crest* → *Materials* → *Upgrade Materials*.

7.2 Migrating from *Crest 4*

Unfortunately, there is no migration path from Crest 4 to 5. This is due to merging three assets together, moved and renamed files, changes in GUIDs and huge changes to serialized data.

Since all GUIDs and file paths have changed, it is possible to import and run both Crest 4 and 5 in one project for manual migration.

When Crest 4 is installed, it adds CREST_OCEAN scripting symbol. When this symbol is present, all Crest 5 components are prefixed with *Crest 5*.

The following tables document items that have been renamed, removed or changed and their new counterpart.

Table 7.1: Components

Crest 4	Crest 5
BoatAlignNormal	FloatingObject
BoatProbes	FloatingObject
OceanDebugGUI	DebugGUI
OceanDepthCache	DepthProbe
OceanPlanarReflections	WaterRenderer[1]
OceanRenderer	WaterRenderer
OceanSampleHeightEvents	QueryEvents
OceanWaterInteraction	N/A[1]
OceanWaterInteractionAdapter	N/A[1]
RegisterAlbedoInput	AlbedoInput
RegisterAnimatedWavesInput	AnimatedWavesInput
RegisterClipSurfaceInput	ClipInput
RegisterDynamicWavesInput	DynamicWavesInput
RegisterFlowInput	FlowInput
RegisterFoamInput	FoamInput
RegisterHeightInput	WaterLevelInput
RegisterSeaFloorDepthInput	WaterDepthInput
RegisterShadowInput	ShadowInput
RegisterAlphaOnSurface	N/A[1]
ShapeGerstnerBatch	N/A[1]
SimpleFloatingObject	FloatingObject
UnderwaterEffect	N/A[1]
UnderwaterEnvironmentalLighting	WaterRenderer[2]
UnderwaterRenderer	WaterRenderer[2]
VisualiseCollisionArea	CollisionAreaVisualizer
VisualiseRayTrace	RayCastVisualizer

[1] Removed

[2] Merged into

Table 7.2: Shaders

Crest 4	Crest 5
CopyDepthBufferIntoCache	N/A[1]
Inputs/AnimatedWaves/GerstnerBatchGeometry	N/A[1]
Inputs/ShapeWaves/SampleSpectrum	Inputs/Waves/AddFromGeometry
Inputs/DynamicWaves/ObjectInteraction	N/A[1]
Inputs/Flow/AddFlowMap	Inputs/Flow/AddFromTexture
Inputs/Foam/AddFromVertColours	Inputs/Foam/AddFromVertexColors
Inputs/Foam/OverrideFoam	Inputs/All/Override
Inputs/All/ScaleByFactor	Inputs/All/Scale
Inputs/AnimatedWaves/GerstnerGeometry	Inputs/ShapeWaves/AddFromGeometry
Inputs/AnimatedWaves/SetBaseWaterHeightUsingGeom	N/A[1]
Inputs/ClipSurface/ConvexHull	WatertightHull[2]
Inputs/ClipSurface/IncludeArea	Inputs/All/Override
Inputs/ClipSurface/RemoveArea	Inputs/All/Override
Inputs/ClipSurface/RemoveAreaTexture	N/A[3]
Inputs/SeaFloorDepth/SetBaseWaterHeightUsingGeomet	Inputs/Level/WaterLevelFromGeometry
Inputs/Depth/OceanDepthFromGeometry	Inputs/Depth/WaterDepthFromGeometry
Inputs/Depth/CachedDepths	N/A[3]
Inputs/Shadows/OverrideShadows	Inputs/All/Override
Inputs/AnimatedWaves/Whirlpool	Whirlpool[4]
Inputs/Flow/Whirlpool	Whirlpool[4]
Ocean	Water
OceanSurfaceAlpha	N/A[1]
UnderwaterCurtain	N/A[1]
UnderwaterMeniscus	N/A[1]

[1] Removed

[2] Replaced with component

[3] Replaced with *Texture Mode*

Table 7.3: Scripting API

Crest 4	Crest 5
SampleHeightHelper	SampleCollisionHelper

Crest 4 never had an official API, thus only the most important changes have been listed. Furthermore, our namespace has changed from *Crest* to *WaveHarmonic.Crest*.

CHAPTER

EIGHT

SAMPLES

8.1 Importing Sample Content

Sample content is hidden in the file system, and requires importing from the package manager window. The *Importing Crest* step needs to be done first.

1. Open *Window → Package Manager*
2. Set the packages menu to *Packages: In Project* (see C below)
3. Select the Crest package you are interested in from the package list under the **Wave Harmonic** group (see H below). If using Unity 6, Crest will show up twice in the menu, make sure to select the package under the **Wave Harmonic** group towards the bottom.
4. In the main window that shows the package's details, find the Samples section (see J below)
5. To import a Sample into your Project, click the *Import into Project* button. This creates a Samples folder in your Project and imports the Sample you selected into it. This is also where Unity imports any future Samples into.

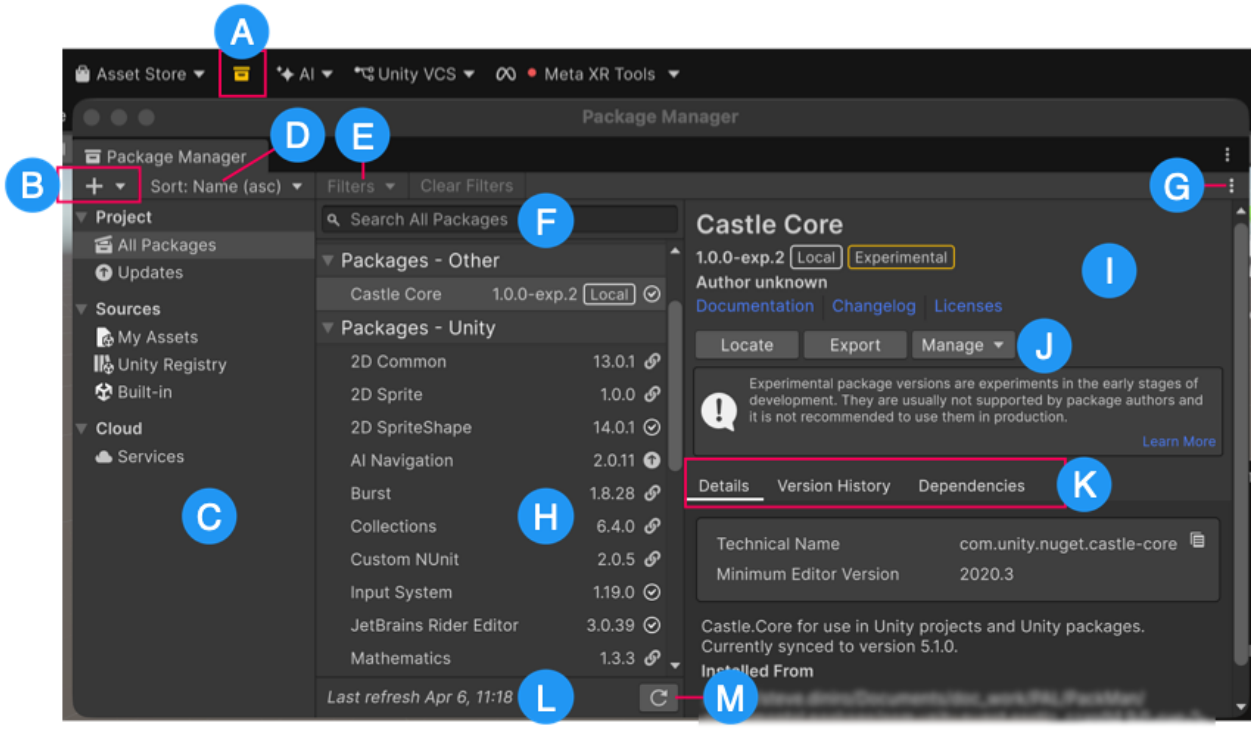


Fig. 8.1: Package Manager window

CHAPTER

NINE

USAGE GUIDE

9.1 Exploration

There are several methods to finding your way through Crest:

- Component Browser: all scripts are under Crest. Furthermore, all scripts are prefixed with Crest for easier searching
- Shader Browser: all shaders are under Crest
- Sample Scenes
- Documentation, especially the *Quick Start*

9.2 Guidance

There are several methods to getting guidance:

- Tooltips
- Validation
- Sample Scenes
- Documentation

The documentation and sample scenes are great at getting an overview or learning the finer points of Crest. We recommend going over the documentation to see what Crest can offer.

We try not to repeat any information that can be best served by tooltips or validation as they can provide immediate guidance to solving problems whilst keeping the documentation digestible. Reading the documentation is the next logical step if either validation or tooltips fall short.

9.3 Editor UI

We have some custom advancements to the UI which are not obvious.

9.3.1 Sliders

Sliders (AKA ranges) have a minimum and maximum value they can represent. Some of our sliders allow to go outside of the slider range by using the accompanying field. There is currently no indication whether it is possible to go outside of the slider range, so some adventuring is required.

CHAPTER

TEN

QUICK START

This section provides a summary of common steps for setting up water with links for further reading:

- **Add water**: Add Crest water to your scene as described in section *Adding Crest to a Scene*.
- **Water surface appearance**: The active water material is displayed below the *Water Renderer* component. See the Appearance section for documentation on water appearance.
- **Add Waves**: Add *Shape FFT* component to a *Game Object* and assign a *Wave Spectrum* asset. Waves can be generated everywhere, or in specific areas. See section *Animated Waves*.
- **Shallow Water**: Reduce waves in shallow water. See section *Shorelines & Shallows*.
- **Oceans, Rivers and Lakes**: Crest supports setting up networks of connected water bodies. See section *Water Bodies*.
- **Underwater**: If the camera needs to go underwater, the underwater effect must be enabled. See section *Underwater*.
- **Dynamic wave simulation**: Simulates dynamic effects like object-water interaction. See section *Dynamic Waves*.
- **Watercraft**: Several components combined can create a convincing watercraft. See page *Floating Objects*.
- **Networking**: Crest is built with networking in mind and can synchronize waves across the network. It also has limited support for headless servers. See section *Network Synchronization*.
- **Open Worlds**: Crest comes with "shifting origin" support to enable large open worlds. See section *Shifting Origin*.

10.1 Adding Crest to a Scene

The default use case for Crest is an infinite ocean. The steps to adding an ocean to a scene are as follows:

- Create a new *Game Object* for the ocean.
  - Assign the *Water Renderer* component to it. This component will generate the water geometry and do all required initialization.
  - Set the Y coordinate of the position to the desired sea level.
- Tag a primary camera as *MainCamera* if one is not tagged already, or provide the *Camera* to the *View Camera* property on the *WaterRenderer* script. If you need to switch between multiple cameras, update the *ViewCamera* field to ensure the water follows the correct view.
- Be sure to generate lighting if necessary. The water lighting takes the ambient intensity from the generated spherical harmonics regardless of whether you use baked or realtime lighting. It can be found at the following:

  Window → *Rendering* → *Lighting Settings* → *Debug Settings* → *Generate Lighting*

> **Tip**
>
> You can check *Auto Generate* to ensure lighting is always generated.

- To add waves, create a new GameObject and add the *Shape FFT* component. See *Animated Waves* section for customization.

CHAPTER

ELEVEN

FREQUENTLY ASKED QUESTIONS

Where is the sample content?

Please see *Importing Sample Content*.

How can I migrate from Crest 4?

Please see *Migrating from Crest 4*.

Is Crest well suited for localized bodies of water such as lakes?

Yes, see *Water Bodies* for documentation.

Can I push the water below the terrain?

Yes, this is demonstrated in Fig. 13.1.

Can I sample the water height at a position from C#?

Yes, see /API/WaveHarmonic.Crest/SampleCollisionHelper, and for more context and examples, see *Queries*. The *WaterRenderer* uses this helper to get the height of the viewer above the water, and makes this viewer height available via the *ViewerHeightAboveWater* property.

Can I trigger something when an object is above or under the water surface without any scripting knowledge?

Yes. Please see *Detecting Above or Below Water*.

How do I disable underwater fog rendering in the scene view?

You can enable/disable rendering in the scene view by toggling fog in the scene view control bar.

Can the density of the fog in the water be reduced?

The density of the fog underwater can be controlled using the *Fog Density* parameter on the water material. This applies to both above water and underwater. The *Depth Fog Density Factor* on the *Underwater Renderer* can reduce the density of the fog for the underwater effect.

Can I remove water from inside my boat?

Yes, this is referred to as 'clipping' and is covered in section *Clip Surface*.

How to implement a swimming character?

As far as we know, existing character controller assets which support swimming do not support waves (they require a volume for the water or physics mesh for the water surface). We have an efficient API to provide water heights, which the character controller could use instead of a physics volume. Please request support for custom water height providers to your favorite character controller asset dev.

Can I render transparent objects underwater?

See *Transparent Object Underwater*.

Can I render transparent objects in front of water?

See *Transparent Object In Front Of Water Surface*.

Can I render transparent objects behind the water surface?

See *Transparent Object Behind The Water Surface*.

11.1 Compatibility

Which platforms does Crest support?

See *Platforms*

Is Crest well suited for medium-to-low powered mobile devices?

Crest is built to be performant by design and has numerous quality/performance levers. However it is also built to be very flexible and powerful and as such can not compete with a minimal, mobile-centric water solution such as the one in the *Boat Attack* project. Therefore we target Crest at PC/console platforms.

That being said, developers have had success with Crest on lower powered platforms like the *Nintendo Switch*. *Apple* devices are good targets as their processors are quite capable all round. *Meta Quest 2* is one device where it will be a struggle to take advantage of Crest.

Preview

Please see *Simple Transparency* and *Quad Mesh Mode* for new, mobile-orientated rendering options.

Can Crest work with networked multiplayer?

Yes, please see *Networked Multiplayer*.

Does Crest support multiple viewpoints like for split-screen multiplayer?

Yes, please see *Split-Screen Multiplayer*.

Does Crest support third-party assets?

Please see *Integrations* for third-party support.

Does Crest support orthographic projection?

Yes. Please see *Orthographic Projection*.

Does Crest support motion vectors?

Yes. Please see *Motion Vectors*.

Does Crest support Depth of Field?

Yes. Please see *Post-Processing*.

Does Crest support Soft Particles?

Yes. Please see *Soft Particles*.

11.2 Troubleshooting

I am seeing errors in the console and/or visual issues

When changing Unity versions, setting up a render pipeline or making changes to packages, the project can appear to break. This may manifest as spurious errors in the log, no water rendering, magenta materials, scripts unassigned in example scenes, etcetera. Often, restarting the Editor fixes it. Additionally, re-importing broken (ie magenta) materials/shaders may be required. Clearing out the *Library* folder can also help to reset the project and clear temporary errors. These issues are not specific to Crest, but we note them anyway as we find our users regularly encounter them.

I am seeing magenta materials after changing render pipelines

Unity has a bug where sometimes it will not correctly switch materials to the current render pipeline. Solutions can vary:

- Restarting Unity
- View the affected material's inspector
- Reimport the affected material/shader

This affects Shader Graph specifically. Find our water Shader Graph, right click and re-import it. It may appear as a blank file icon instead of the usual Shader Graph icon.

I can enter play mode, but errors appear in the log at runtime that mention missing 'kernels'

Recent versions of Unity have a bug that makes shader import unreliable. Please try reimporting the *Packages/Crest/Runtime/Shaders* folder using the right click menu in the project view. Or simply close Unity, delete the Library folder and restart which will trigger everything to reimport.

Why aren't my prefab mode edits not reflected in the scene view?

Crest does not support running in prefab mode which means dirty state in prefab mode will not be reflected in the scene view. Save the prefab to see the changes.

I am seeing "Crest does not support OpenGL/WebGL backends." in the editor

It is likely Unity has defaulted to using OpenGL on your platform. You will need to switch to a supported graphics API like Vulkan. You will need to make Vulkan the default by overriding the graphics APIs.

Why I am seeing "The referenced script on this Behavior is missing!" or similar in Crest's sample scenes and prefabs?

This is normal and can be ignored. The sample scenes support all render pipelines which require render pipeline specific components to be serialized in the scene. If a render pipeline package is missing, then those components will also be missing.

There is staircasing and/or small bumps on the water surface

This is due to lack of precision. Set *Water Renderer* → *Simulations* → *Animated Waves* → *Texture Format Mode* to Precision, and set *Water Renderer* → *Simulations* → *Water Level* → *Texture Format Mode* to Automatic.

If the problem persists, and you are targeting a mobile platform, then enable *Project Settings* → *Crest* → *Full Precision Displacement On Half Precision Platforms*.

If the problem still persists, and you are using mesh and/or splines inputs, then it is likely due to mesh aliasing. A solution is blurring the LOD targets which can be enabled at *Water Renderer* → *Simulations* → *Water Level* → *Blur*.

For splines specifically, increasing the mesh density by increasing Subdivisions may also be required. This can be done for the Water Level Input only by overriding the settings.

> **Tip**
>
> These artifacts can be hidden well enough with waves and/or normal maps which can save some performance.

I am seeing water and/or waves jitter at the shoreline

This can happen if TAA (Temporal Anti-Aliasing) or similar temporal effects are active. Make sure motion vectors are enabled. Please see *Motion Vectors* for more information.

Part III

Basics

CHAPTER

TWELVE

UNDERWATER

Crest supports seamless transitions above/below water. It can also have a meniscus which renders a subtle line at the intersection between the camera lens and the water to visually help the transition. This is demonstrated in the *Main.unity* scene in the example content.

For performance reasons, the underwater effect is disabled if the viewpoint is not underwater.

> **Tip**
>
> Use opaque or alpha test materials for underwater surfaces. Transparent materials do not receive the underwater effect. This can be solved with *Integrate Water Volume (Shader Graph)*.

12.1 Underwater Renderer

The Underwater Renderer is built into the Water Renderer and executes a fullscreen underwater effect before the transparent pass. Transparent objects by default do not receive the underwater effect.

> **Tip**
>
> You can enable/disable rendering in the scene view by toggling fog in the scene view control bar.

12.1.1 Setup

- Configure the water material for underwater rendering. Under *Surface Options* make sure both sides are enabled with either *Render Face* to *Both* or *Double-Sided* enabled.
- Enable *Crest Water* → *Underwater* → *Enabled*.
- Make sure that *Crest* is set to the underwater material.

> **Note**
>
> If you are using the underwater effect in URP, it is recommended to set *Opaque Downsampling* to *None*. *Opaque Downsampling* will make everything appear at a lower resolution when underwater. Be sure to test to see if recommendation is suitable for your project.

12.1.2 Appearance

The underwater effect gets its appearance from two sources: the water material and the underwater material.

The water material can be overridden by setting *Water Renderer → Volume Material* to a material variant of the water material. Not every property on the water material affects the underwater effect. To see what can be overridden, see the read-only properties on the underwater material.

The underwater material has more properties which can further adjust the underwater effect.

12.1.3 Integrate Water Volume (Shader Graph)

Transparent objects do not receive the underwater effect due to the effect rendering before the transparent pass. We provide a Shader Graph node, *Integrate Water Volume*, to conditionally apply underwater fog.

Additionally, this can exclude Unity's own fog when underwater (for transparent and opaque).

12.2 Meniscus Renderer

The Meniscus Renderer is integrated into the Water Renderer under the Meniscus foldout. It renders a meniscus effect where the surface intersects the near plane (or in case of portals, the edge of the portal).

To configure the meniscus effect, there are options under foldout, and the material inspector appears at the bottom of the Water Renderer component.

12.3 Detecting Above or Below Water

The Water Renderer component has the *Viewer Height Above Water* property which can be accessed with `WaterRenderer.Instance.ViewerHeightAboveWater`. It will return the signed height from the water surface of the camera rendering the water.

There is also the *Query Events* component which uses UnityEvents to provide a script-less approach to triggering changes.

12.4 Portals & Volumes

Underwater rendering can be restricted to a mesh.

To help with this feature, the Portals package is available as a separate purchase and its offline manual is available in the package.

12.5 Underwater Only

The underwater effect can render without the surface to save performance. Simply disable *Water Renderer → Surface → Enabled*.

Furthermore, disable all of the simulations to save performance further.

12.6 Legacy Underwater

For those who need the older implementation of the underwater/meniscus effect, enable *Project Settings → Crest → Legacy Underwater*. It may take a minute for recompilation to begin.

The legacy effect typically has worst performance, and there should not be a need to use it. It renders after the transparent pass, which means transparent objects will receive the underwater effect without an integration, albeit incorrectly.

CHAPTER

THIRTEEN

WATER INPUTS

Inputs provides a means for developers to control the various simulations powering Crest. The following video covers the basics:

https://www.youtube.com/watch?v=sQIakAjSq4Y

Fig. 13.1: Crest 4: Basics of Adding Inputs (outdated but still useful)

13.1 Inputs

Each simulation target has an associated generic input:

- *Absorption Input*
- *Albedo Input*
- *Animated Waves Input*
- *Clip Input*
- *Dynamic Waves Input*
- *Flow Input*
- *Foam Input*
- *Scattering Input*
- *Shadow Input*
- *Water Depth Input*
- *Water Level Input*

There are also inputs which are more specific like the Sphere Water Interaction.

13.2 Input Modes

Inputs (eg Flow Input) can have multiple authoring modes. Support for modes will vary across inputs. More complicated inputs will have their own component like the Sphere Water Interaction.

13.2.1 Geometry Mode

This mode is the same as using the *Renderer Mode* with a Mesh Renderer, except much easier to set up. All you need to provide is a mesh and change the Transform to your needs.

13.2.2 Global Mode

Global is applied everywhere.

13.2.3 Texture Mode

This mode uses a compute shader to apply your texture to the target simulation. It performs better than shader equivalents by reducing draw calls as typically a shader needs to do one draw call per LOD (eg if LOD Count is seven then that is seven draw calls per input).

This has great utility for customization and integration with Crest, as it abstracts away the complexity of the system if you treat the texture as an intermediary between your own simulations/effects and Crest's.

13.2.4 Spline Mode

To help with this feature, the Splines package is available as a separate purchase and its offline manual is available in the package.

13.2.5 Renderer Mode

This is the most advanced type of input and allows rendering any geometry/shader into the water system data. One could draw foam directly into the foam data, or inject a flow map baked from an offline simulation.

The geometry can come from a Mesh Renderer, or it can come from any Renderer component such as a Trail Renderer, Line Renderer or Particle System. This geometry will be rendered from a orthographic top down perspective to "print" the data onto the water. For simple cases, it is recommended to use an upwards facing quad for the best performance.

> **Note**
>
> It is recommended to use unlit shader templates or unlit *Shader Graph* for data if not using one of ours. If using Shader Graph with HDRP, then you must use the Built-In target in your Shader Graph.

The following shaders can be used with any input:

- **Override** sets the data to a value.
- **Scale** scales the water data between zero and one inclusive. It is multiplicative, which can be inverted, so zero becomes no data and one leaves the data unchanged.
- **Utility** a very configurable shader exposing blend modes and operations.

13.2.6 Paint Mode

To help with this feature, the Paint package is available as a separate purchase and its offline manual is available in the package.

CHAPTER

FOURTEEN

WATER EXCLUSION

Features detailed here can either exclude or include the surface and/or volume.

14.1 Clip Surface

https://www.youtube.com/watch?v=jXphUy__J0o

Fig. 14.1: Crest 4: Water Bodies and Surface Clipping (outdated but still useful)

This data drives clipping of the water surface (has no effect on the volume), as in carving out holes. It is similar to Unity's Terrain Holes feature. This can be useful for hollow vessels or low terrain that goes below sea level. Data can come from primitives (signed-distance), geometry (convex hulls) or a texture.

To turn on this feature, enable the *Surface Clipping* simulation on the *WaterRenderer* script, and ensure the *Alpha Clipping* is enabled on the water material.

The data contains 0-1 values. Holes are carved into the surface when the value is greater than 0.5.

14.1.1 User Inputs

The Clip Input component can write data to the simulation and supports the *Primitive Mode*, *Texture Mode*, *Spline Mode*, *Paint Mode* and *Renderer Mode*.

Primitive

Clip areas can be added using signed-distance primitives which produce accurate clipping and supports overlapping. The position, rotation and dimensions of the primitive is determined by the Transform.

Renderer

The following input materials are provided:

- **Clip Include Area**: Removes clipping data so the water surface renders.
- **Clip Remove Area**: Adds clipping data to remove the water surface.

14.2 Displacement

Displacing the water surface is another option for removing water from an area. Simply push the water below the surface using an Animated Waves Input or Water Level input.

When using the Animated Waves Input, it is possible to displace the water surface and have it affect underwater rendering:

- Enable the Displacement collision layer (see *Collision Layers*)
- Set Displacement Pass to *Lod Independent (Last)*.
- Set Collision Layer on Floating Objects to anything earlier to Everything

> **Tip**
>
> Displacement is not as precise as the clip simulation. It requires thicker walls to hide the edge of the effect.

It is also possible to nest buoyant objects, in addition to above:

- Set Collision Layer to not Everything on Floating Objects that you want to nested

14.3 Watertight Hull

The Watertight Hull component uses the clip simulation and/or displacement to remove water from a convex hull. Simply assign a convex hull mesh.

14.4 Mask Underwater

The *Portals & Volumes* feature can remove both the water surface and the underwater volume. Otherwise, enable/disable the Underwater Renderer where needed.

14.5 Remove Water In Area

The Water Override can add and remove the Surface, Volume and Physics from a given area.

CHAPTER

FIFTEEN

WATER BODIES

15.1 Oceans

By default Crest generates an infinite body of water at a fixed sea level, suitable for oceans and very large lakes.

15.2 Lakes

Crest can be configured to efficiently generate smaller bodies of water, using the following mechanisms.

- The waves can be generated in a limited area - see the *User Inputs* section.
- If the lake altitude differs from the global sea level, use a *Water Level Input* to change the height of the water to match. It is recommended to cover a larger area than the lake itself, to give a protective margin against lower mesh densities from LODs in the distance.
- For advanced lake authoring, see the *Local Overrides*.

15.3 Streams

There are several ways to accomplish streams. The most pertinent data is flow for the current and height for the stream gradient (optional).

15.3.1 Flow & Height Map

A river being a large stream, often without a visible gradient, can be implemented with just a *Flow Map*. The water level can also be adjusted to make a stream gradient by using a height map or a mesh with the Water Level Input.

15.3.2 Splines

Splines are a great option for creating streams. They can direct flow and create gradients.

To help with this feature, the Splines package is available as a separate purchase and its offline manual is available in the package.

15.3.3 Shallow Water Simulation

The Shallow Water Simulation can simulate streams and bake the final output to a texture.

To help with this feature, the Shallow Water package is available as a separate purchase and its offline manual is available in the package.

15.4 Separate Water Bodies

The Water material has the Custom Mesh toggle which configures the material for usage with separate mesh renderers. This can be optimal for water bodies which are separate and may need different material properties.

There are a few considerations:

- There will be no displacement from our system (ie waves etc) when using this mode, but displacement will still contribute to normals.
- Currently, inputs like Sphere Water Interaction will not work with them, unless the inputs are placed at sea level, as they are unaware of these separate bodies

15.5 Troubleshooting

Ocean waves are affecting rivers and/or lakes

This is typically caused by incorrect blending, input feathering or large global waves with chop. For the latter, chop will add horizontal displacement which can cause waves to displace onto another input, circumnavigating the input's blending.

The following are potential solutions:

- If varying the water level, enable *Shape FFT/Gerstner → Sea Level Only* on the ShapeFFT/Gerstner used for ocean waves. This will fade global waves so there are none one meter or further away from sea level. This is enabled by default.
- Make sure *Shape FFT/Gerstner → Blend* on the non global input is set to an option which overrides previous values. Also make sure to pay attention to *Shape FFT/Gerstner → Queue*, to ensure local waves execute after global ones.
- *Shape FFT/Gerstner → Feather* will feather the edges of the input which affects blending.
- Using water depth for wave attenuation can help greatly with reducing global waves affecting inland water bodies
  - Be sure to check *Water Renderer → Simulations → Water Depth → Attenuation In Shallows*, *Shape FFT/Gerstner → Waves → Respect Shallow Water Attenuation* and *Respect Shallow Water Attenuation* on the Wave Spectrum, as it reduces the effectiveness of attenuation.
- If changing wave attenuation is not desired, expanding the input to give a buffer is a good solution. For splines, override the spline radius at *Shape FFT/Gerstner → Waves → Mode → Radius* for the ShapeFFT/Gerstner attached to the spline.

CHAPTER

SIXTEEN

FLOATING OBJECTS

Crest is capable of supporting various floating objects from barrels to watercraft.

16.1 Physics

Floating Object handles all physics interactions including buoyancy and drag. It has two models available: Align Normal and Probes.

Align Normal is a simple buoyancy model that attempts to match the object position and rotation with the surface height and normal. This can work well enough for small watercraft that do not need perfect floating behavior, or floating objects such as buoys, barrels, etc. Furthermore, this model is less affected by physics, like mass, making authoring much easier like adding additional colliders.

Probes is a more advanced implementation that computes buoyancy forces at a number of Probes and uses these to apply force and torque to the object. This gives more accurate results at the cost of more queries.

16.1.1 Usage

Steps to get started:

1. Add a Floating Object to your Rigidbody.
2. Choose a model on the Floating Object.
    1. Add probes if using the Probes model.
3. Configure the Floating Object.
    - Force Strength is the most important value for keeping the object afloat. This value will differ greatly depending on the chosen model. Probes will take the rigid body mass into account. Align Normal will not and requires balancing Force Strength with Height Offset to get desired results.
    - Drag and Object Width/Length are important for keeping the object stable.

16.1.2 Troubleshooting

If a floating object is behaving erratically, then there could be an internal collision. Review your collider hierarchy and see if it can be simplified.

16.2 Movement

Several scripts exist for adding movement to floating objects to make watercraft.

16.2.1 Usage

- Add a *Controller* to a *Floating Object*
- Add a control to the Controller

16.2.2 Controller

Controller is a general purpose watercraft controller script. It can control thrust, steer and dive mechanics by adding forces to the rigid body directly.

16.2.3 Controls

A control is an abstraction to allow input from various sources. It simply provides an XYZ value which maps to steer, dive and thrust respectively. We provide a few controls in the package.

Example

For a player control example, import the Boats sample. The Player Control works with both old and new Unity input systems. There is also a dive capable control in the Submarine sample. These are only examples because they are not the ideal usage of Unity's input systems.

The idea is that developers can easily make their own controls by extending the Control class. Input could come from the Unity input system or from some mechanic in the game world like a lever.

Fixed Control

Simply provides a constant input which is configurable from the editor.

16.3 Interactions

The Sphere Water Interaction component is used to add interactions to floating objects like wakes. See *Adding Interaction Forces* section for more information on this component.

16.4 Watertightness

There are various methods to removing water from Crest detailed on the *Water Exclusion* page.

16.5 Troubleshooting

Wakes are causing Watercraft to jitter or behaving slightly erratically

When using the Sphere Water Interaction for wakes, it can create a feedback loop which causes watercraft to jitter. Excluding dynamic waves for this object is currently the only solution to solving these jitters. Please see *Collision Layers* for more information.

CHAPTER

SEVENTEEN

EVENTS

17.1 Query Events

The purpose of this component is to provide Unity Events for a script-less approach to triggering changes using water information from queries.

It can perform the following:

- **Above/Below Water:** invoke a UnityEvent when the attached game object is above or below the water surface - once per state change. A common use case is to use it to trigger different audio when above or below the surface.
- **Distance From Surface**: Constantly sends the vertical distance from the water surface. Can be used to fade in and out audio.
- **Distance From Edge**: Constantly sends the horizontal distance from the water's edge. Can be used to fade in and out shoreline audio.

Part IV

Appearance

CHAPTER

EIGHTEEN

ALBEDO

The Albedo feature allows a color layer to be composited on top of the water surface. This is useful for projecting color onto the surface like duckweed.

This is somewhat similar to decals, except the color only affects the water.

Note

HDRP has a Decal Projector feature that works with the water shader, and the effect is more configurable and may be preferred over this feature. When using decals, be sure to enable Affects Transparent.

URP 2022 has a decal system, but it does not support transparent surfaces like water yet.

18.1 User Inputs

The Albedo Input component can write data to the simulation and supports the *Renderer Mode*.

Any geometry or particle system can add color to the water. It will be projected from a top down perspective onto the water surface.

The following shaders are available under *Crest/Inputs/Albedo* if using the *Renderer Mode*:

- **Color** a basic shader for writing a color or texture to the Albedo data including blend modes.

Note

If using a shader with multiple passes (or Shader Graph), if the output looks incorrect then change the Shader Pass Index.

CHAPTER

NINETEEN

BIOLUMINESCENCE

Bioluminescence is driven by the *Foam* simulation, as the foam simulation is a proxy for turbulence which triggers bioluminescence in real life.

19.1 Usage

The set up bioluminescence from the defaults, do the following:

- Enable *Water Material* → *Surface Inputs* → *Bioluminescence* → *Foam Bioluminescence Enabled*

If it is not showing, then check the following:

- Enable *Water Renderer* → *Simulations* → *Foam* → *Enabled*
- Enable *Water Material* → *Surface Inputs* → *Foam* → *Foam Enabled*
- Make sure there is something actually generating foam in the scene (eg choppy waves, shorelines etc)

19.2 Customization

All the properties for customizing bioluminescence is under *Water Material* → *Surface Inputs* → *Bioluminescence*.

The sparkles can be customized further by editing the green channel of the foam texture.

CHAPTER

TWENTY

COLOR

The Absorption and Scattering targets allows changing the water volume color at the texel level. These are separate simulations, but are often used together.

They both contain depth-based absorption/scattering, which can simulate suspended matter, dissolved matter and other scattering elements close to shore.

20.1 User Inputs

The Absorption/Scattering Input component can write data to the simulation and supports the *Texture Mode*, *Spline Mode*, *Paint Mode* and *Renderer Mode*.

20.1.1 Texture

Be aware that the absorption simulation stores a computed absorption value, not the absorption color you will see on materials and scripts. This is an issue when using this mode, as the input texture needs to be in the computed value. To calculate the absorption value, see /API/WaveHarmonic.Crest/WaterRenderer/CalculateAbsorptionValueFromColor.

20.1.2 Renderer

The following shaders are available under *Crest/Inputs/Absorption* and *Crest/Inputs/Scattering* if using the *Renderer Mode*:

- **Color** a basic shader for writing a color.

20.2 Material

If you need to change the absorption value via script, then do the following:

```
var material = WaterRenderer.Instance.Material;
var color = Color.red; // Replace with your color.
material.SetColor(Shader.PropertyToID("_Crest_AbsorptionColor"), color);
material.SetVector(Shader.PropertyToID("_Crest_Absorption"), WaterRenderer.
↪CalculateAbsorptionValueFromColor(color));
```

CHAPTER

TWENTYONE

FOAM

Crest simulates foam generation by choppy water (ie *pinched* wave crests) and in shallow water to approximate foam from splashes at the shoreline. Each update (default is 30 updates per second), the foam values are reduced to model gradual dissipation of foam over time.

To turn on this feature, enable *Water Renderer* → *Simulations* → *Foam* → *Enabled*.

To configure the foam simulation, create a Foam Lod Settings with *Assets* → *Create* → *Crest* → *Simulation Settings* → *Foam Sim Settings*, and assigning it to the Water Renderer component in your scene.

Tip

Clicking "New" next to *Water Renderer* → *Simulations* → *Foam* → *Settings* will create a new one for you.

21.1 Simulation Settings

Configuring how foam generates can be done in *Water Renderer* → *Simulations* → *Foam* → *Settings*. Each setting has descriptive tooltips. This is an overview of each category of foam generation.

The most important setting is the *Foam Fade Rate*. It applies to all foam in the simulation, including shoreline foam, which means it needs to be balanced with the following settings.

Furthermore, the Maximum is also important. This setting can be the key to unlocking lasting wake foam.

21.1.1 Whitecaps

Crest detects where waves are 'pinched' and deposits foam to approximate whitecaps. Pinched waves comes from *Chop* in for *Animated Waves* and the *Horizontal Displace* setting for *Dynamic Waves*.

21.1.2 Shoreline Foam

If water depth is provided to the system (see *Depth Simulation*), the foam simulation can automatically generate foam when water is very shallow, which can approximate accumulation of foam at shorelines.

21.2 User Inputs

Crest supports inputting foam data into the system, which can be helpful for fine tuning where foam is placed.

The Foam Input component can write data to the simulation and supports the *Texture Mode*, *Spline Mode*, *Paint Mode* and *Renderer Mode*. The *Texture Mode* is the most efficient mode.

The following shaders are available under *Crest/Inputs/Foam* if using the *Renderer Mode*:

- **Add From Texture** adds foam values read from a user provided texture. Can be useful for placing 'blobs' of foam as desired, or can be moved around at runtime to paint foam into the simulation. This is an alternative to the *Texture Mode* as it can provide more properties to adjust.
- **Add From Vertex Colors** can be applied to geometry and uses the red channel of vertex colors to add foam to the simulation. Similar in purpose to *Add From Texture*, but can be authored in a modeling workflow instead of requiring a texture.

21.3 Foam Shading

Foam shading is configured on the water surface material.

There are two foam textures, one with bioluminescence, and one without. If you want sparkles in foam-based bioluminescence, then make sure you are using the packed foam texture.

For a more stylized look, try reducing the Foam Feather value, or provide a stylized foam texture.

21.4 Troubleshooting

How do I reduce pixelation when viewed up close?

Find the foam textures and change the compression quality. Removing compression will eliminate pixelation. Be wary that this will increase performance cost.

They can be found in: `Packages/com.waveharmonic.crest/Runtime/Textures`.

CHAPTER

TWENTYTWO

LIGHTING

As other shaders would, the water will get most its lighting from the primary directional light (ie sun or moon).

See *Chunk Template* for advanced configuration.

22.1 Reflections

See the dedicated *Reflections* page.

22.2 Refractions

Refractions sample from the camera's color texture. Anything rendered in the transparent pass or higher will not be included in refractions.

See *Transparent Object In Front Of Water Surface* for issues with Crest and other refractive materials.

22.3 Chunk Template

Crest uses Mesh Renderers to render chunks of water throughout the scene. Mesh Renderers have many settings for configuring how a mesh responds to different lighting components like probes.

The Chunk Template allows you to provide a pre-configured Mesh Renderer to override Crest's defaults. Some settings cannot be overridden as they make no sense or require support from Crest.

To use this feature, create a prefab with a Mesh Renderer. The only other requirement is to **not** have a Water Chunk Renderer present in the prefab.

> **Tip**
>
> Clicking "New" next to *Water Renderer* → *Surface Renderer* → *Chunk Template* will create a new one for you.

Settings which cannot be overridden are:

- Receive Shadows
- Motion Vectors

Other settings may not have any effect depending on the level of support with the render pipeline.

Alternatively, there is an event which is raised after chunk creation: /API/WaveHarmonic.Crest/SurfaceRenderer/OnCreateChunkRendere

CHAPTER

TWENTYTHREE

REFLECTIONS

Reflections contribute significantly to the appearance of the water. The look of the water will dramatically changed based on the reflection environment.

Crest uses Shader Graph which makes it, by default, ready to receive all the various sources of reflections Unity provides:

- Global Reflections
- Reflection Probes
- Screen Space Reflections
- Planar Reflection Probes

By default, Crest receives reflections from the global cubemap which only contains the skybox.

Furthermore, Crest provides *Planar Reflections* for both above and below the water (known as TIR) for all render pipelines.

23.1 Planar Reflections

Crest has planar reflections for both above and below the water (known as TIR) for all render pipelines.

23.1.1 Usage

For above water reflections:

- Enable *Water Renderer → Reflections → Enabled*
- Enable *Water Material → Surface Inputs → Reflections (Planar) → Planar Reflections Enabled*
- Set *Water Renderer → Reflections → Layers* to the layers you want in the reflections

For below water reflections (TIR):

- Complete the above steps first.
- Set *Water Renderer → Reflections → Mode* to either Below or Both.
- Set *Water Material → Surface Inputs → Reflections (Underwater) → Occlusion (U)* to 1
- Optionally, set *Water Material → Surface Inputs → Reflections (Underwater) → Total Internal Reflection Intensity (U)* to 1 for full reflections

23.1.2 Configuration

Planar reflections can be configured both from two foldouts:

- *Water Renderer → Reflections*
- *Water Material → Surface Inputs → Reflections (Planar)*

Additionally, TIR can be further configured in another foldout:

- *Water Material → Surface Inputs → Reflections (Underwater)*

23.1.3 Performance

Enabling planar reflections for both above and below water can be very expensive. It is only recommended to have one enabled.

All of the properties in the *Water Renderer → Reflections* foldout are mostly performance related. They will have descriptive tooltips.

23.2 Reflection Probes

Crest is set up to receive reflections from probes by default, but further configuration is possible. Please see *Chunk Template* for more information.

> **Tip**
>
> When configuring your reflection probes for water, consider Box Projection and Probe Blending on both your probes and render pipeline settings.

CHAPTER

TWENTYFOUR

SHADOWS

The shadow data consists of two channels. One is for normal shadows (hard shadow term) as would be used to block specular reflection of the light. The other is a much softer shadowing value (soft shadow term) that can approximate variation in light scattering in the water volume.

This data is captured from the shadow maps Unity renders before the transparent pass. These shadow maps are always rendered in front of the viewer. The Shadow simulation then reads these shadow maps and copies shadow information into its LOD textures.

To turn on this feature, enable *Water Renderer* → *Simulations* → *Shadows* → *Enabled*.

24.1 User Inputs

The Shadow Input component can write data to the simulation and supports the *Renderer Mode*. There are currently no shadow specific shaders, but the general purpose shaders under *Crest/Inputs/All* will accomplish most things (especially Override).

24.2 Simulation Settings

The shadow simulation can be configured under *Water Renderer* → *Simulations* → *Shadows*.

In particular, the soft shadows are very soft by default, and may not appear for small/thin shadow casters. This can be configured using the *Jitter Diameter Soft* setting.

There will be times when the shadow jitter settings will cause shadows or light to leak. An example of this is when trying to create a dark room during daylight. At the edges of the room the jittering will cause the water on the inside of the room (shadowed) to sample outside of the room (not shadowed) resulting in light at the edges. Reducing the *Jitter Diameter Soft* setting can solve this, but we have also provided a *Shadow Input* component which can override the shadow data. This component bypasses jittering and gives you full control.

CHAPTER

TWENTYFIVE

STYLIZATION

Crest has several controls on the water material pertinent to stylization.

Example

Check out the Stylized *sample* which has most of the following advice already applied.

Stylized water will often have simpler details to give a clean look to match the clean and simpler look of the scene. This can be done by muting normals and giving foam a more defined, toon shape.

The water surface material has the following properties which help the most with stylization:

- *Normals* → *Normal Strength*: This affects the strength of the final normal. Reducing it can give a simpler, cleaner look (consider increasing refractions if lowering).
- *Normals* → *Normal Map Enabled*: Provides finer detail which can be disabled.
- *Foam* → *Foam Feather*: Reduce to give a more defined, toon shape.
- *Caustics* → *Caustics Grey Point*: reducing this value to reduce darkening.
- *Reflections* → *Fresnel*: Changes reflectivity grazing angles.
- *Reflections (Underwater)* → *Total Internal Reflection Intensity*: Reducing this value gives the surface a more see-through look when looking through the surface from underwater.
- *Reflections* → *Fresnel*: Changes reflectivity grazing angles.
- *Volume Lighting* → *Absorption Color*: The alpha channel will change visibility below water.
- *Subscurface Scattering* → *SSS Intensity*: Increasing this value will increase scattering color at wave peaks.

Furthermore, the material has texture slots for more significant customization:

- *Normals* → *Normal Map*
- *Foam* → *Foam Texture*
- *Caustics* → *Caustics Texture*

Lastly, there is also the underwater material:

- Density Factor: Decrease to increase visibility when underwater
- Out-Scattering X: Reducing these will reduce the darkening effect when underwater

All of this is for a more clear and reflective look. Alternatively, you could go the other way with water which focuses on a strong the water color.

There are many other properties on the water material to adjust the look of the water - each having tooltips worth reading!

Part V

Simulation

CHAPTER

TWENTYSIX

ANIMATED WAVES

The Animated Waves simulation contains the animated surface shape. This typically contains the waves from shape components, but can also contain waves from the Dynamic Waves simulation. All waves will eventually be combined into this simulation so the water shader only needs to sample once to animate vertices.

26.1 Environmental Waves

The ShapeFFT and ShapeGerstner components are used to generate waves in Crest.

ShapeFFT is great at generating waves with a varied appearance.

For advanced situations where a high level of control is required over the wave shape, the Shape Gerstner component can be used to add specific wave components. It can be especially useful for swells and shoreline waves. See the *Shoreline Waves* section for more information on the latter.

26.1.1 Wave Conditions

The appearance and shape of the waves is determined by a *Wave Spectrum*. A default wave spectrum will be created if none is specified. To author wave conditions, click the *New* or *Clone* button next to the *Spectrum* field. The resulting spectrum can then be edited by expanding this field.

The spectrum can be freely edited in Edit mode, and is locked by default in Play mode to save evaluating the spectrum every frame (this optimization can be disabled using the *Spectrum Fixed At Runtime* toggle). The spectrum has sliders for each wavelength to control contribution of different scales of waves. To control the contribution of 2m wavelengths, use the slider labelled '2'. Note that the wind speed may need to be increased on the *Water Renderer* component in order for large wavelengths to be visible.

There is also control over how aligned waves are to the wind direction. This is controlled via the Wind Turbulence control on the Shape FFT component.

Another key control is the Chop parameter which scales the horizontal displacement. Higher chop gives crisper wave crests but can result in self-intersections or 'inversions' if set too high, so it needs to be balanced.

To aid in tweaking the spectrum, we provide a standard empirical wave spectrum model from the literature, called the 'Pierson-Moskowitz' model. To apply this model to a spectrum, select it in the *Empirical Spectra* section of the spectrum editor which will lock the spectrum to this model. The model can be disabled afterwards which will unlock the spectrum power sliders for hand tweaking.

> **Tip**
>
> Notice how the empirical spectrum places the power slider handles along a line. This is typical of real world wave conditions which will have linear power spectrums on average. However actual conditions can vary significantly

based on wind conditions, land masses, etc, and we encourage experimentation to obtain visually interesting wave conditions, or conditions that work best for gameplay.

The waves will be dampened/attenuated in shallow water if a Depth simulation is used (see *Depth Simulation*). The amount that waves are attenuated is configurable using the Attenuation In Shallows setting.

Together these controls give the flexibility to express the great variation one can observe in real world seascapes.

Wind

There are global wind controls on the Water Renderer which includes WindZone support. Global wind can be overridden on each Shape* component.

26.1.2 User Inputs

Waves can be applied everywhere in the world, placed along or orthogonal to a spline, or injected via a custom shader. The Shape FFT/Gerstner components supports the *Global Mode*, *Texture Mode*, *Spline Mode*, *Paint Mode* and *Renderer Mode*.

Global

This is the default mode and places waves globally. Global waves are additive and multiple global wave components can be used together. Furthermore, transition between wave conditions by interpolating between the Weight property of two Shape FFT/Gerstner components.

Renderer

The following shaders are available under *Crest/Inputs/Shape Waves* if using the *Renderer Mode*:

- **Add From Geometry** places waves from the spectrum into the world confined to the provided mesh.

26.2 Animated Waves

The environmental waves are termed "Animated Waves" in Crest. The Animated Waves simulation is also the displacement target where any data for vertex displacement is stored.

26.2.1 User Inputs

The Animated Waves Input component can write data to the simulation and supports the *Renderer Mode*.

If wanting to add waves, you should use the Shape FFT/Gerstner components as detailed above. This input is more for wave manipulation, manipulating displacement or custom waves via a custom shader.

For the Animated Waves Input's *Renderer Mode*, the following shaders are provided under the shader category *Crest/Inputs/Animated Waves*:

- **Push Water Under Convex Hull** pushes the water underneath the geometry. Can be used to define a volume of space which should stay 'dry'.
- **Wave Particle** is a 'bump' of water. Many bumps can be combined to make interesting effects such as wakes for boats or choppy water. Based loosely on http://www.cemyuksel.com/research/waveparticles/.

Under *Crest/Inputs/All* there is the following:

- **Scale By Factor** scales the waves by a factor where zero is no waves and one leaves waves unchanged. Useful for reducing waves.

26.2.2 Advanced Settings

The environmental waves are termed "Animated Waves" in the Crest system and can be configured under *Water Renderer → Simulations → Animated Waves*. Properties have detailed tooltips.

26.3 Shoreline Wave Simulation

The shallow water simulation is a new, next-generation feature for Crest and is not required for shoreline waves.

To help with this feature, the Shallow Water package is available as a separate purchase and its offline manual is available in the package.

26.4 Troubleshooting

I am seeing water and/or waves jitter at the shoreline

This can happen if TAA or similar temporal effects are active. Make sure motion vectors are enabled. Please see *Motion Vectors* for more information.

Ocean waves are affecting rivers and/or lakes

This is typically caused by incorrect blending, input feathering or large global waves with chop. For the latter, chop will add horizontal displacement which can cause waves to displace onto another input, circumnavigating the input's blending.

The following are potential solutions:

- If varying the water level, enable *Shape FFT/Gerstner → Sea Level Only* on the ShapeFFT/Gerstner used for ocean waves. This will fade global waves so there are none one meter or further away from sea level. This is enabled by default.
- Make sure *Shape FFT/Gerstner → Blend* on the non global input is set to an option which overrides previous values. Also make sure to pay attention to *Shape FFT/Gerstner → Queue*, to ensure local waves execute after global ones.
- *Shape FFT/Gerstner → Feather* will feather the edges of the input which affects blending.
- Using water depth for wave attenuation can help greatly with reducing global waves affecting inland water bodies
  - Be sure to check *Water Renderer → Simulations → Water Depth → Attenuation In Shallows*, *Shape FFT/Gerstner → Waves → Respect Shallow Water Attenuation* and *Respect Shallow Water Attenuation* on the Wave Spectrum, as it reduces the effectiveness of attenuation.
- If changing wave attenuation is not desired, expanding the input to give a buffer is a good solution. For splines, override the spline radius at *Shape FFT/Gerstner → Waves → Mode → Radius* for the ShapeFFT/Gerstner attached to the spline.

I cannot create large waves or storms

Check that wind is not limiting the wave spectrum. Increasing wind will allow the spectrum to be fully developed. Sources of wind in order:

- *Wind Speed* on the same Shape component (if overridden)
- Either *Wind Speed* or *Wind Zone* (if set) under *Water Renderer → Environment*

CHAPTER

TWENTYSEVEN

DYNAMIC WAVES

Environmental/animated waves are 'static' in that they are not influenced by objects interacting with the water. 'Dynamic' waves are generated from a multi-resolution simulation that can take such interactions into account.

To turn on this feature, enable *Water Renderer → Simulations → Dynamic Waves → Enabled*. To configure the simulation, create or assign a *Dynamic Wave Lod Settings* asset to Settings.

The dynamic wave simulation is added on top of the animated waves to give the final shape.

The dynamic wave simulation is not suitable for use further than approximately 10km from the origin. At this kind of distance the stability of the simulation can be compromised. Use Shifting Origin to avoid traveling far distances from the world origin.

27.1 Adding Interaction Forces

Dynamic ripples from interacting objects can be generated by placing one or more spheres under the object to approximate the object's shape. To do so, attach one or more Sphere Water Interaction components to children on the object and set the *Radius* parameter to roughly match the shape.

Non-spherical objects can be approximated with multiple spheres, for an example see the *Spinner* object in the Boat sample scene which is composed of multiple sphere interactions. The intensity of the interaction can be scaled using the *Weight* setting. For an example of usages in boats, see the provided prefabs with the Boat sample.

27.2 Simulation Settings

The simulation can be configured with a *Dynamic Wave Lod Settings* asset.

The key settings that impact stability of the simulation are the **Damping** and **Courant Number** settings.

The Debug GUI overlay reports the number of simulation steps taken each frame.

27.3 User Inputs

The Dynamic Waves Input component can write data to the simulation and supports the *Renderer Mode*.

The recommended approach to injecting forces into the dynamic wave simulation is to use the Sphere Water Interaction component as described above. This component will compute a robust interaction force between a sphere and the water, and multiple spheres can be composed to model non-spherical shapes.

However for when more control is required, custom forces can be injected directly into the simulation using the *Renderer Mode*. The following input shader is provided under *Crest/Inputs/Dynamic Waves*:

- **Dampen Circle** dampens dynamic waves within a circle.

CHAPTER

TWENTYEIGHT

WATER LEVEL

The Water Level (ie height) simulation can simulate water bodies of varied height and tides. The default water level is sea level, which is the height of the Water Renderer.

28.1 Tides

It is possible to move the entire water surface on the Y axis to simulate tides. The Water Level simulation can be used to localize the tide.

28.2 User Inputs

28.2.1 Geometry

Sets the water level using the provided geometry.

See *Geometry Mode* for more.

28.2.2 Spline

The spline input mode will use the spline mesh to set the water level.

See *Spline Mode* for more.

28.2.3 Paint

The paint input mode will enable paint tools to paint add/set the water level.

The water level can be raised or lowered with `LeftClick` and `Control-LeftClick` respectively. Use `Shift-LeftClick` to remove water level.

See *Paint Mode* for more.

28.2.4 Texture

Set water level using a height map.

See *Texture Mode* for more.

28.2.5 Renderer

Set the water level using a mesh using the *Water Level From Geometry* material.

See *Renderer Mode* for more.

28.3 Troubleshooting

There is staircasing and/or small bumps on the water surface

This is due to lack of precision. Set *Water Renderer* → *Simulations* → *Animated Waves* → *Texture Format Mode* to Precision, and set *Water Renderer* → *Simulations* → *Water Level* → *Texture Format Mode* to Automatic.

If the problem persists, and you are targeting a mobile platform, then enable *Project Settings* → *Crest* → *Full Precision Displacement On Half Precision Platforms*.

If the problem still persists, and you are using mesh and/or splines inputs, then it is likely due to mesh aliasing. A solution is blurring the LOD targets which can be enabled at *Water Renderer* → *Simulations* → *Water Level* → *Blur*.

For splines specifically, increasing the mesh density by increasing Subdivisions may also be required. This can be done for the Water Level Input only by overriding the settings.

> **Tip**
>
> These artifacts can be hidden well enough with waves and/or normal maps which can save some performance.

CHAPTER

TWENTYNINE

SHORELINES & SHALLOWS

Crest uses water depth information for various purposes, from attenuating large waves in shallow water to generating foam near shorelines. The way this information is typically generated is through the Depth Probe component, which takes one or more layers and renders everything in those layers (and within its bounds) from a top-down orthographic view to generate a height field for the seabed. These layers could contain the render geometry/terrains, or it could be geometry that is placed in a non-rendered layer that serves only to populate the probe. By default, this generation is done at run-time during startup, but the component exposes other options, such as generating offline and saving to an asset or rendering on demand.

The seabed affects the wave simulation in a physical way - the rule of thumb is that waves will be affected by the seabed when the water depth is less than half of their wavelength. For example when the water is 250m deep, this will start to dampen 500m wavelengths from the spectrum, so it is recommended that the seabed drop down to at least 500m away from islands so there is a smooth transition between shallow and deep water without a 'step' in the sea floor which appears as a discontinuity in the surface waves and/or a line of foam. Alternatively, there is *Water Renderer* → *Simulations* → *Water Depth* → *Shallows Maximum Depth* which smooths the attenuation to a provided maximum depth where waves will be at full strength.

29.1 Depth Simulation

This simulation stores information that can be used to calculate the water depth. Specifically it stores the terrain height, which can then be differenced with the sea level to obtain the water depth. This is useful information to the system; it is used to attenuate large waves in shallow water and to generate foam near shorelines. It is calculated by rendering the geometry in the scene for each LOD, from a top-down perspective, and recording the Y value of the surface.

The following will contribute to water depth:

- Any Unity Terrain if *Water Renderer* → *Simulations* → *Water Depth* → *Include Terrain Height* is enabled.
- Objects captured by a Depth Probe. The scene depth will be rendered into this probe once, and the results are saved. Once the probe is populated, it is then copied into the Water Depth LOD Data. More details further below.
- Objects that have the *Depth Input* component attached. These objects will render every frame. This is useful for any dynamically moving surfaces.

When the water is e.g. 250m deep, this will start to dampen 500m wavelengths, so it is recommended that the sea floor drop down to around this depth away from islands so that there is a smooth transition between shallow and deep water without a visible boundary.

29.1.1 Setup

https://www.youtube.com/watch?v=jcmqUlboTUk

Fig. 29.1: Crest 4: Depth Probe (formerly Ocean Depth Cache) usage and setup (outdated but still useful)

> **Tip**
>
> By default, Crest will include terrain height automatically. This can be disabled with *Water Renderer → Simulations → Water Depth → Include Terrain Height*. The aforementioned option is less accurate than a DepthProbe, and will not include details like rocks, but reduces friction for beginners.

One way to inform Crest of the seabed is to attach the Depth Input component. Crest will record the height of these objects every frame, so they can be dynamic.

The Main sample scene has an example of a probe set up around the island. The probe Game Object is called *Island-DepthCache* and has a Depth Probe component attached. The following are the key points of its configuration:

- The transform position X and Z are centered over the island
- The transform position y value is set to the sea level
- The transform scale is set to 540 which sets the size of the probe. If gizmos are visible and the probe is selected, the area is demarcated with a white rectangle.
- The Capture Range is the minimum and maximum height of any surfaces above the sea level that will render into the probe. If gizmos are visible and the probe is selected, this cutoff is visualized as a translucent gray rectangle.
- The Layers field contains the layer that the island is assigned to (*Terrain* in our project). Only objects in these layer(s) will render into the probe.
- Both the transform scale (white rectangle) and the Layers property determine what will be rendered into the probe.

By default the probe is populated in the *Start()* function. It can instead be configured to populate from script by setting the Refresh Mode to On Demand and calling the /API/WaveHarmonic.Crest/DepthProbe/Populate method on the component from script.

Once populated the probe contents can be saved to disk by clicking the *Bake* button.

Overhangs

Since the Depth Probe captures from a top-down perspective, overhangs can cause problems by being captured instead of capturing what is below.

To solve this use the Capture Range to remove the overhang. The next potential issue is the hole left behind by cutting the top of the captured object. The Fill Holes feature can alleviate this by filling only the hole left behind. Set the *Fill Holes Capture Height* which relative to the Capture Range (ie 100 is 100 above the second)

Dynamic Depth Probes

When authoring large worlds, it is convenient to have the Depth Probe follow the camera instead of manually placing them. This will have a significant performance overhead, but there are ways to mitigate the cost.

- Set Placement to Viewer
- Disable *Signed Distance Fields → Generate*, as it can be expensive
- Change settings in *Quality Settings Override*.
- Set the Layer to exclude the terrain, and only include what is necessary
  - Enable *Water Renderer → Simulations → Water Depth → Include Terrain Height*. This will sample the height directly from the terrain efficiently. It will not include terrain trees or details (objects)

This approach will save on rendering the terrain an extra time which can be significant.

29.1.2 Shoreline Foam

Once the Water Depth is running, shoreline foam can be configured. See *Simulation Settings* section for more information.

29.1.3 Troubleshooting

Crest runs validation on the depth probes - look for warnings/errors in the Inspector, and in the log at run-time, where many issues will be highlighted. To run validation, click the Validate Setup button at the bottom of the Water Renderer component inspector.

To inspect the contents of the probe, there is a preview pane available at the bottom.

29.2 Shoreline Waves

29.2.1 Attenuation

Modeling realistic shoreline waves efficiently is a challenging, open problem. We discuss further and make suggestions on how to set up shorelines using global waves with Crest in the following video.

https://www.youtube.com/watch?v=Y7ny8pKzWMk

Fig. 29.2: Crest 4: Tweaking Shorelines (outdated but still useful)

29.2.2 Wave Map

Shape FFT/Gerstner components can accept a texture using the *Texture Mode*. The X/Y values map to the X/Z direction of the waves and their magnitude is the intensity of the waves.

29.2.3 Splines

Alternatively, using Shape Gerstner with a spline is an effective way to create shoreline waves. You will need to set Reverse Wave Weight to zero to avoid waves also going in the opposite direction and set Blend to Blend which effectively overwrites existing waves to prevent global waves from interfering.

To help with this feature, the Splines package is available as a separate purchase and its offline manual is available in the package.

29.2.4 Simulation

To help with this feature, the Shallow Water package is available as a separate purchase and its offline manual is available in the package.

29.3 Troubleshooting

I am seeing water and/or waves jitter at the shoreline

This can happen if TAA or similar temporal effects are active. Make sure motion vectors are enabled. Please see *Motion Vectors* for more information.

CHAPTER

THIRTY

FLOW

Flow is the horizontal motion of the water volume. It does not affect wave directions, but transports the waves horizontally. This horizontal motion also affects physics. Furthermore, flow also affects foam, normals and caustics.

This can be used to simulate water currents and other water flow.

30.1 User Inputs

Crest supports adding any flow velocities to the system. The Flow Input supports the following modes:

30.1.1 Spline

Flow will be generated along the spline direction. Reverse the spline if going in the wrong direction. The velocity of the flow can be adjusted per point with the *Spline Point Flow Data*.

See *Spline Mode* for more.

30.1.2 Paint

The paint input mode will enable paint tools to paint flow in the direction and magnitude of the brush stroke.

Use `LeftClick` and `Shift-LeftClick` to add and remove flow respectively.

See *Paint Mode* for more.

30.1.3 Texture

Writes a flow texture (ie flow map) into the system. It expects the XZ velocity to be packed into the RG components respectively. Furthermore if Normalized is enabled then it expects the values to be in the 0-1 range where 0.5 is zero velocity.

See *Texture Mode* for more.

30.1.4 Renderer

The following input shaders are provided under *Crest/Inputs/Flow*:

- **Add Flow Map** - The same as *Texture* except with more options.
- **Fixed Direction** - Adds flow in a fixed direction for the entire input.

See *Renderer Mode* for more.

Part VI

Advanced

CHAPTER

THIRTYONE

LEVEL OF DETAIL

There are a small number of parameters that control the construction of the water shape and geometry. All the information is in tooltips under *Water Renderer → Level of Detail*.

There are a few important things to note:

- Resolution will apply to each simulation, unless they override it (some will by default).
- Mesh density is based on both Resolution and *Geometry Down Sample Factor*. The latter is applied after the former. Increasing Resolution will increase density, and increasing *Geometry Down Sample Factor* will decrease density.
- Please see *Multiple Viewpoints* for more information for *Center of Detail → Multiple Viewpoints*.

31.1 Multiple Viewpoints

Preview

This feature is in preview and may be subject to changes or needs refinement.

A viewpoint is not the same thing as a camera. A viewpoint is a center of detail, which is typically rendered at the camera's position. Crest will render features like underwater based on the camera, but the surface is rendered based on the viewpoint.

Crest's LOD system can support multiple viewpoints. This needs to be enabled with *Water Renderer → Level of Detail → Center of Detail → Multiple Viewpoints*.

The LOD system will now render in the camera loop, per-camera. By default, all cameras not tagged as MainCamera are excluded as viewpoints. Multiple cameras can be tagged as MainCamera, but this may cause problems, as it is expected for only one camera to be the main camera. Managing the exclusion list can be done at: *Water Renderer → Level of Detail → Center of Detail → Camera Exclusions*.

If *Non Main Camera* is remove from the exclusions, then any game camera which has the Water layer in its culling mask will have its own viewpoint.

Adding a Water Camera component alongside the Camera component will bypass the exclusions and mark the camera for having its own viewpoint.

There is also an option to keep viewpoints updated in the background (*Water Renderer → Level of Detail → Center of Detail → Data Background Mode*) - even when the camera is no longer rendering. Typically, this should not be required, but there are cases where you may want the viewpoint primed like when switching multiple cameras which are far enough apart.

CHAPTER

THIRTYTWO

LOCAL OVERRIDES

The Local Override component can add and remove the Surface, Volume and Physics from a given area. It can also override the materials for the Surface and Volume.

It operates on the CPU which means it cannot be as granular as a simulation.

32.1 Usage

The Local Override component, if present, marks areas of the scene where water should be present or not. It can be created by attaching this component to a Game Object and setting the X/Z scale to set the size of the water body. If gizmos are enabled, an outline showing the size will be drawn in the Scene View.

1. Add Local Override component to a GameObject.
2. Position and scale the GameObject so the bounds covers the desired area (bounds visible with gizmos).
3. If overriding the volume and/or physics, the Vertical option can be used to also heck the Y axis.
4. Configure exclusion (*Include/Exclude Water*) and *Material Overrides* as need.

Tip

If you need the ocean removed completely, the component shows two global settings: Default Clipping State and Default Excludes. The former requires *Precision* to be enabled.

32.2 Include/Exclude Water

It is possible to include/exclude the Surface, Volume and Physics from a given area.

The surface removal operation happens at chunk level. Volume and Physics are done precisely. For Physics, only objects that check /API/WaveHarmonic.Crest/WaterRenderer/HasWater are affected.

32.2.1 Precision

The Local Override component turns off chunks that do not overlap the desired area. The Clip Surface feature can be used to precisely remove any remaining water inside or outside the intended area. Enable Precise to use the clip surface with the Local Override. Additionally, the clipping system can be configured to clip everything by default, and then areas can be defined where water should be included. See the *Clip Surface* section.

32.3 Material Overrides

The Local Override component can override the water material on the water chunks. This can be used to give closed lakes a distinct appearance.

Important

It is important to understand that since this feature cannot be applied partially to a water chunk, and a water chunk can overlap two water bodies. Thus, this feature does not work well with bordering water bodies - including bordering an ocean. Typically it works best with only a single lake in the scene.

It can also override the volume materials. Unlike overriding the surface material detailed above, this is very precise. Be wary that there is no transition from one local override to another.

Tip

Crest has many simulations which can change the appearance of water at the texel level. We encourage everyone to consider those options instead.

CHAPTER

THIRTYTHREE

QUERIES

33.1 Usage

All providers which can be queried use a similar interface. They also have an accompanying helper class (known as sample helpers) to make basic queries simpler.

The providers built into our system perform queries asynchronously; queries are offloaded to the GPU or to spare CPU cores for processing. This has a few non-trivial impacts on how the query API must be used.

Firstly, queries need to be registered with an ID so that the results can be tracked and retrieved later. This ID needs to be globally unique, and therefore should be acquired by calling *GetHashCode()* on an object/component which will be guaranteed to be unique. A primary reason why sample helpers are useful is that they are objects themselves and therefore can pass their own ID, hiding this complexity from the user.

Secondly, even if only a one-time query is needed, the query function should be called every frame until it indicates that the results were successfully retrieved. Querying a second time, in the same frame, using the same ID, will stomp over the last query points. Posting the query and polling for its result are done through the same function.

Finally, due to the above properties, the number of query points posted from a particular owner should be kept consistent across frames. The helper classes always submit a fixed number of points for the current frame, so satisfy this criteria.

Important

For each unique query ID:

- Queries should only be made once per frame.
- Number of query points posted should be kept consistent across frames.

The following code example uses the SampleCollisionHelper to query the water height at the current position. Since all the helpers have a similar interface, it can be applied to others with only some parameter changes.

```
using UnityEngine;
using WaveHarmonic.Crest;

public class ScriptExample : MonoBehaviour
{
   SampleCollisionHelper helper = new();

   // To avoid a one frame delay, call this before the WaterRenderer.LateUpdate call.
   void Update()
   {
      // Call only once per frame.
```

(continues on next page)

(continued from previous page)

```
        if (helper.SampleHeight(transform.position, out float height))
        {
            Debug.Log($"Height is {height}.");
        }
    }
}
```

> **Tip**
>
> For the most user-friendly, use-case driven approach, which requires not scripting, there is the *Query Events* component.

Alternatively, if you wish to query the provider directly, then this is how:

```
using UnityEngine;
using WaveHarmonic.Crest;

public class ScriptExample : MonoBehaviour
{
    readonly Vector3[] _query = new Vector3[1];
    readonly Vector3[] _height = new Vector3[1];

    // To avoid a one frame delay, call this before the WaterRenderer.LateUpdate call.
    void Update()
    {
        if (WaterRenderer.Instance == null) return;

        var provider = WaterRenderer.Instance.AnimatedWavesLod.Provider;

        _query[0] = transform.position;

        // Call only once per frame.
        var status = provider.Query(GetHashCode(), 0, _query, _height, null, null);

        if (provider.RetrieveSucceeded(status))
        {
            Debug.Log($"Height is {_height[0]}.");
        }
    }
}
```

The advantage with the direct method is if you to query multiple points at a time (we are only querying a single point in the example). This is more efficient using this approach.

33.2 Collision Shape

The system has a few paths for computing information about the water surface such as height, displacement, flow and surface velocity. These paths are covered in the following subsections, and are configured with *Water Renderer* → *Simulations* → *Animated Waves* → *Collision Source* dropdown.

The system supports sampling the collision shape at different resolutions. The query functions have a parameter,

Minimum Length, which is used to indicate how much detail is desired. Wavelengths smaller than half of this minimum spatial length will be excluded from consideration.

To simplify the code required to get the water height, or other data via scripting, the /API/WaveHarmonic.Crest/SampleCollisionHelper is provided.

Research

We use a technique called *Fixed Point Iteration* to calculate the water height. We gave a talk at GDC about this technique which may be useful to learn more: https://www.gdcvault.com/play/1023011/Fixed-Point-Iteration-A-Simple.

The *Collision Area Visualizer* debug component is useful for visualizing the collision shape for comparison against the render surface. It draws debug line crosses in the Scene View around the position of the component.

33.2.1 GPU Queries

This is the default and recommended choice for when a GPU is present. Query positions are uploaded to a compute shader, which then samples the water data and returns the desired results. The result of the query accurately tracks the height of the surface, including all wave components, depth caches and other Crest features. Set *Water Renderer* → *Simulations* → *Animated Waves* → *Collision Source* to GPU.

Collision Layers

GPU queries support collision layers. Collision layers allow excluding collision sources by querying the displacement data at certain points.

For example, using the After Animated Waves layer includes all waves, but excludes Dynamic Waves. This has often been requires as when using the Sphere Water Interaction to produce wakes, it can cause a feedback loop and jitters.

Collision layers can be enabled with *Water Renderer* → *Simulations* → *Animated Waves* → *Collision Layers*. When enabling a layer it will allow that layer to be included or excluded. On any object which queries collisions, like Floating Object, set the layer to what you want to include. The layers are in order (except Everything), and selecting a layer will include everything preceding it (eg After Dynamic Waves also includes Animated Waves).

Example

To have a Floating Object exclude dynamic waves, make sure Dynamic Waves layer is enabled on Collision Layers, and then set *Floating Object* → *Collision Layer* to After Animated Waves.

See the boats in the Boats sample.

There is also the Displacement layer which is a layer at the end. It can be used to render inputs which affect underwater and have nested buoyant objects (eg floating barrel in a ship).

33.2.2 CPU Queries

On platforms where a GPU is not present, or for authoritative servers, CPU queries are available.

To help with this feature, the Portals package is available as a separate purchase and its offline manual is available in the package.

33.3 Flow

Flow can also be queried, which is currently done on the GPU only. Query against the *Flow Provider* or use the /API/WaveHarmonic.Crest/SampleFlowHelper.

33.4 Water Depth

Currently, the water depth cannot be sampled, but the distance to water's edge can. See /API/WaveHarmonic.Crest/SampleDepthHelper/SampleDistanceToWaterEdge.

CHAPTER

THIRTYFOUR

MULTIPLAYER

34.1 Split-Screen Multiplayer

Preview

This feature is in preview and may be subject to changes or needs refinement.

Split-Screen typically require having *Multiple Viewpoints*, otherwise when one player moves away from the other, they both will not receive the same visual quality of the water surface.

34.2 Networked Multiplayer

Networked multiplayer is possible with *Network Synchronization*.

If the server needs the water shape to run physics, but does not have a GPU, then we have a CPU path: see *CPU Queries*. Different server conditions can be emulated in Editor using the Force Batch Mode and Force No GPU toggles on the Water Renderer.

Note that *Dynamic Waves* are not synchronized across the network and should not be relied upon in multiplayer projects.

CHAPTER

THIRTYFIVE

TIME CONTROL

By default, Crest uses the current game time given by Time.time when simulating and rendering the water. In some situations it is useful to control this time, such as an in-game pause or to synchronize wave conditions over a network. This is achieved through what we call *TimeProviders*, and a few use cases are described below.

> **Note**
>
> The *Dynamic Waves* simulation must progress frame by frame and can not be set to use a specific time, and also cannot be synchronized accurately over a network.

35.1 Supporting Pause

One way to pause time is to set Time.timeScale to 0. In many cases it is desirable to leave Time.timeScale untouched so that animations continue to play, and instead pause only the water. To achieve this, attach a Custom Time Provider component to a GameObject and assign it to *Water Renderer* → *General* → *Time Provider*. Then time can be paused by setting the *_paused* variable on the Custom Time Provider component to *false*.

The Custom Time Provider also allows driving any time to the system which may give more flexibility for specific use cases.

A final alternative option is to create a new class that implements the *ITimeProvider* interface and call *WaterRenderer.Instance.PushTimeProvider()* to apply it to the system.

35.2 Network Synchronization

A requirement in networked games is to have a common sense of time across all clients. This can be specified using an offset between the clients Time.time and that of a server.

This is supported by attaching a *TimeProviderNetworked.cs* component to a GameObject, assigning it to *Water Renderer* → *General* → *Time Provider*, and at run-time setting *TimeProviderNetworked.TimeOffsetToServer* to the time difference between the client and the server.

If using the Mirror network system, set this property to the network time offset. Crest expects that if the client time is ahead then the offset needs to be a negative value which may mean that you need to invert the sign of the offset first.

Please see *Networked Multiplayer* for more multiplayer information.

35.3 Timelines and Cutscenes

One use case for this is for cutscenes/timelines when the waves conditions must be known in advance and repeatable. For this case you may attach a Cutscene Time Provider component to a Game Object and assign it to *Water Renderer* → *General* → *Time Provider*. This component will take the time from a Playable Director component which plays a cutscene Timeline. Alternatively, a Custom Time Provider component can be used to feed any time into the system, and this time value can be keyframed, giving complete control over timing.

CHAPTER

THIRTYSIX

OPEN WORLDS

Crest follows the camera so it is inherently suitable for open worlds, but there will be engine issues to account for.

36.1 Shifting Origin

To help with this feature, the Shifting Origin package is available as a separate purchase and its offline manual is available in the package.

CHAPTER

THIRTYSEVEN

RENDERING

37.1 Camera Filtering

Some features of the Water Renderer (eg Surface, Underwater, Meniscus) will have a Layer and a Camera Exclusion field for filtering cameras. Cameras will be filtered according to the following:

- Have the appropriate *Layer* in its culling mask
- Not part of the *Camera Exclusion* rules or has a *Water Camera* component

Together, they make powerful filtering rules for multiple camera use cases.

37.2 Injection Point

By default, where Crest will render in the render pipeline is determined by the *Surface Type* on the material (either Opaque or Transparent). The Surface Type is set to Transparent by default, as opaque water is typically not required. *Injection Point* provides more options.

37.2.1 Before Transparency

Preview

This feature is experimental, and it may have incompatibilities not listed here.

Overriding the injection point using *Water Renderer* → *Rendering* → *Injection Point* with *Before Transparency* will unlock more advanced rendering options, and can improve compatibility with other rendering features:

- Available for other refractive objects (when Write Color is enabled)
- Soft Particles support (when Write Depth is enabled)

Warning

Due to a limitation with Unity, when using this feature with BIRP, the water surface will not receiving lighting from additional lights, nor support advanced lighting features like light cookies.

Warning

Currently, this feature does not support MSAA when using HDRP.

> **Warning**
>
> Due to a bug with Unity, HDRP motion vectors are not compatible.

37.3 Orthographic Projection

Crest supports orthographic projection, but it might require some configuration to get a desired appearance.

Crest uses the camera's position for the LOD system which can be awkward for orthographic which uses the size property on the camera. Use the *Viewpoint* property on the *Water Renderer* to override the camera's position. Increasing the minimum scale (*Water Renderer* → *Level of Detail* → *Scale*) will help spread the detail out to make the water appear more uniform. Pick a minimum scale that looks best.

Adjusting the camera's Y value is important in preventing a hard seam running across the surface. The camera's height will need to be reasonable as per the size value.

Underwater only works with a positive near clip plane.

37.4 Motion Vectors

Crest supports motion vectors which is often required for many rendering features like TAA, upscaling and even SSR (Screen-Space Reflections).

These should be enabled by default, but to enable motion vector support:

- Enable *Water Renderer* → *Surface Renderer* → *Motion Vectors*
- For HDRP:
  - Start with Motion vectors for transparent objects
  - Enable *Transparent Writes Motion Vectors* on the water material

> **Note**
>
> URP requires Unity 6. If you are using URP, and have upgraded from an earlier version of Unity to Unity 6, you will need to re-import the water Shader Graph from the package manager again for motion vectors to work.

> **Warning**
>
> Due to a bug in Unity, motion vectors for BIRP incurs an additional render pass of the water surface (on top of what is already required for motion vectors).

37.5 Post-Processing

The water surface writes to the depth buffer, and supports copying the water from the depth buffer into the depth texture for post-processing effects like Depth of Field.

These should be enabled by default, but to enable depth write for post-processing:

- On the water material, enable *Surface Options* → *Depth Write* to set to Force Enabled
- For BIRP and URP:

- Enable *Water Renderer → Surface Renderer → Write Depth*. This will copy the depth buffer into the depth texture after the transparent pass (it could include more than just water)

Warning

Due to a limitation with Unity, when using this feature with BIRP, the water surface vertex shader needs to be rendered an additional time, thus incurring a more significant performance cost.

37.6 Soft Particles

Soft particles use the scene depth to blend particles to avoid a hard cutoff where particles intersect with the scene. The water surface depth is not available when soft particles render, thus there will be a hard cutoff.

Preview

Please see the in-preview feature, *Injection Point* , for a proper solution to this problem.

CHAPTER

THIRTYEIGHT

SCRIPTING

38.1 Resources

Crest does not use the Resources folder. Instead a scriptable object which holds the references to all our compute and internal shaders is stored on the Water Renderer. This will tell Unity to include these files in a standalone build.

If instantiating the Water Renderer via scripting in builds, and there is no reference to a Water Renderer in the scene (including via prefab), then you must add a reference to the Resources asset in a scene to include all the necessary assets in the build. The Resources asset will not include shaders that are exposed in the shader selection for materials.

38.2 Scripting API

There is a separate API document provided in the same folder as this manual. Each package will have its own manual and API document.

38.2.1 Water Singleton

If you need a reference to the WaterRenderer, it can be obtained via the singleton WaterRenderer.Instance.

38.2.2 Assemblies

When using Assembly Definitions, there are two extra assemblies that need to be referenced: Shared and Scripting. The former is required to compile correctly, while the latter is optional.

The Scripting assembly provides extension methods necessary to adding input components:

```
// Add the foam input with the texture mode.
var input = gameObject.AddComponent<FoamLodInput>(LodInputMode.Texture);
// Retrieve the texture mode data and assign a texture.
var data = input.GetData<FoamTextureLodInputData>();
data.Texture = texture;
```

CHAPTER

THIRTYNINE

INTEGRATIONS

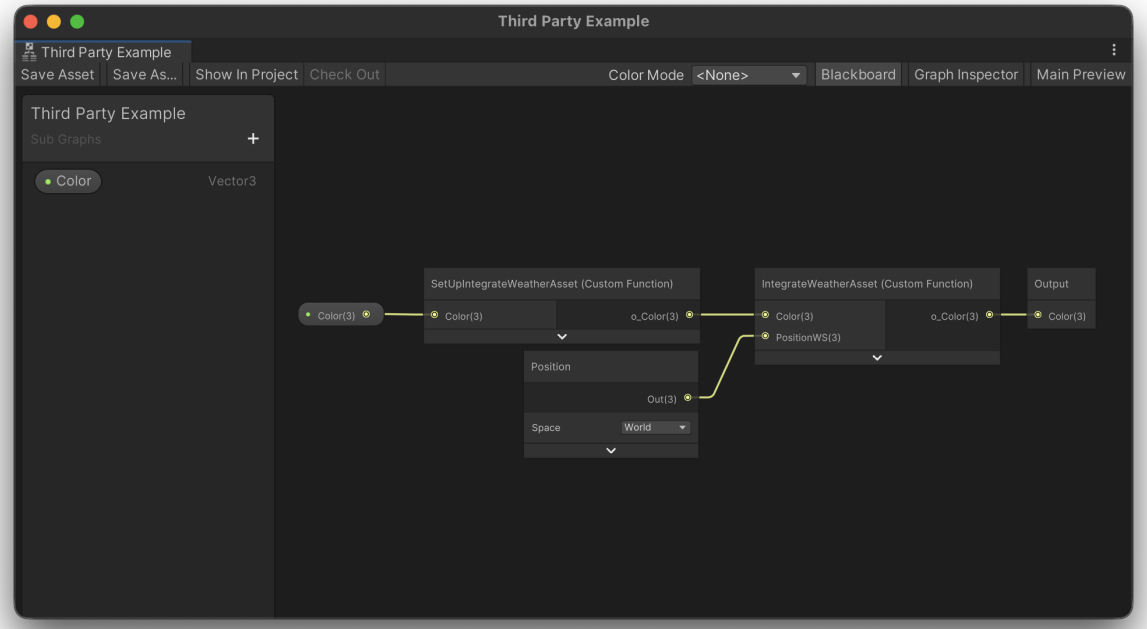
This page is for those who are looking to integrate their asset with Crest.

39.1 Atmospheric Scattering And Fog

39.1.1 Manual Integration

We provide a method to manipulate the final output of the shader without having to convert the Shader Graph.

1. Create a subgraph which takes a Vector3 input and Vector3 output
2. Create two Custom Function Nodes inside the subgraph
    1. Both need to take a Vector3 input and Vector3 output, and pass them through. This prevents the compiler from stripping your code.
3. The first node needs to define static variables for anything you need from the Shader Graph
4. The second node needs to assign those static variables with the Shader Graph properties required by the integration
5. Define `CREST_INTEGRATE_COLOR_AFTER_FOG` in the second node which performs the integration. This will run after Unity's fog, but only for above water. There is also `CREST_INTEGRATE_COLOR_FINAL` which runs last regardless.
6. See *Integrating Third-Party Sub-Graph* for the final steps



The following examples show the entire contents of the Custom Function Node (including the open-ended functions).

> **Tip**
>
> We only care about the signature of our macros, thus feel free to rename anything else to your liking.

An example of the SetUpIntegrateWeatherAsset node:

```
o_Color = Color; }

// If the integration needs properties from Shader Graph, you will need to define a
// static variable for each one here:
static float3 IntegrateWeatherAsset_PositionWS;

void SetUpIntegrateWeatherAsset_Close() {
```

An example of the IntegrateWeatherAsset node:

```
   o_Color = Color;

   // Assign properties from the Shader Graph to the static variables defined earlier:
   IntegrateWeatherAsset_PositionWS = PositionWS;
}

#ifndef SHADERGRAPH_PREVIEW

// Place any includes required by the third party here:
#include "<path-to-include>.hlsl"
```

(continues on next page)

(continued from previous page)

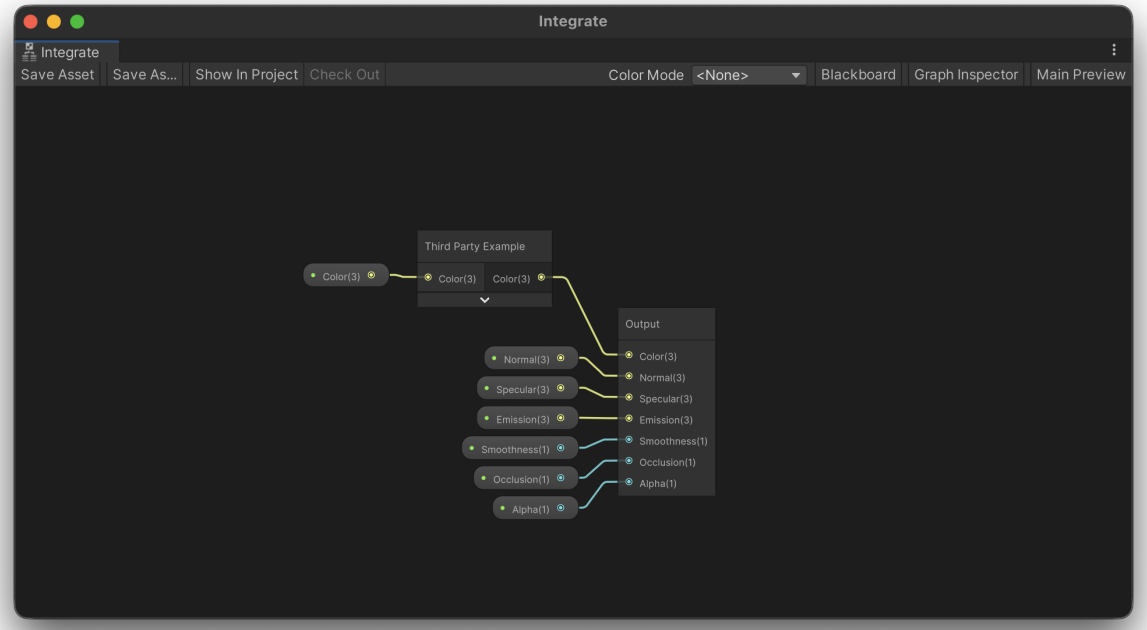
```
// Define the CREST_INTEGRATE_COLOR_AFTER_FOG macro,
// which will run only for above water, here:
#define CREST_INTEGRATE_COLOR_AFTER_FOG(color) \
color.rgb = ApplyFog(color.rgb, IntegrateWeatherAsset_PositionWS);

// There is also the CREST_INTEGRATE_COLOR_FINAL (same signature) which runs at the
// end regardless.

#endif // SHADERGRAPH_PREVIEW

void IntegrateWeatherAsset_Close() {
```

39.2 Integrating Third-Party Sub-Graph

If a third party has provided a sub graph to integrate with Crest, then please do the following:

1. Add their node inside the `Packages/com.waveharmonic.crest/Runtime/Shaders/Surface/Graph/Fragment/Integrate.shadersubgraph`. The reason to integrate here instead of directly in the main graph, is that this will less likely to be updated by us.
2. Connect through all properties from the Blackboard to their sub graph, and then to the Output node.

The following example has an integration which only needs the color passed through.



Tip

When updating Crest, uncheck this file to avoid the importer replacing it. If this file needs updating, we will mention it as a breaking change in the release notes.

Part VII

Guides

CHAPTER

FORTY

PLATFORMS

Testing occurs primarily on MacOS and Windows. iOS and Android are also periodically tested.

Firstly, make sure your target platform adheres to the *Requirements*.

We have users targeting the following platforms:

- Windows
- MacOS
- Linux/SteamDeck *
- PlayStation *
- Xbox *
- Switch * **
- iOS **
- Android/Quest **
- Web

* We do not have access to these platforms ourselves.
** Performance is a challenge on these platforms. Please see the *Performance Guide*.

Crest also supports VR/XR Multi-Pass, Multi-View and Single Pass Instanced rendering.

> **Tip**
>
> See FAQs under the *Compatibility* heading for more information.

40.1 Meta Quest

40.1.1 Requirements

- Vulkan

40.1.2 Usage

Follow Meta's and/or Unity's instructions to get set up with Quest and Unity development.

> **Tip**
>
> Start with Unity's VR template in latest LTS to avoid performance issues.

Make sure that Vulkan is the only one in the Auto Graphics API list to avoid issues. Be wary that if using the Oculus XR plugin, every time it is enabled in the XR project window it will add and set OpenGL as the graphics API.

40.2 Web

> **Preview**
>
> Web support is experimental.

40.2.1 Requirements

- WebGPU
- Browser supporting WebGPU. See WebGPU support
- Browser supporting texture-formats-tier2. See texture-formats-tier2 support

40.2.2 Usage

The most likely problem that will occur in a build is the hard limit on texture sampling. If the water surface does not render, check the browser console. If you see the following warning, then you need to disable features on the water surface material:

> The number of sampled textures ... in the Fragment stage exceeds the maximum per-stage limit (16).

Under *Project Settings* → *Crest* → *Features* there are per-platform feature toggles. Disabling features here will reduce the sample count.

On the water material, the Flow toggle will contribute to the texture count. Furthermore, toggles/dropdowns under *Surface Options/Parameters* can also contribute, especially *Alpha Clip* and *Receive Shadows*.

Lastly, having the Portals package installed will contribute several sampled textures to the total limit - even if not used (unless *Alpha Clip* is disabled).

40.2.3 Limitations

The following is known to be not working:

- *Water Renderer* → *Simulations* → *Water Depth* → *Enable Signed-Distance Fields*
- *Crest/Water* → *Surface Options* → *Receive Shadows* for BIRP

These known limitations may be solved in future versions.

40.2.4 Troubleshooting

The water surface is not rendering

Firstly, read the paragraphs under *Usage*. Then check the browser console, and if you see errors or warnings other than the one described above, please let us know.

Not all browsers work as expected

See the requirements.

40.3 Xbox

If you receive the following error:

> Crest requires graphic devices that support compute shaders

Place DirectX 12 at the top of the list of the Graphics API list. For UWP apps, ensure that your app is declared as Game.

CHAPTER

FORTYONE

PERFORMANCE GUIDE

The foundation of Crest is architected for performance from the ground up with an innovative LOD system. It is tweaked to achieve a good balance between quality and performance in the general case, but getting the most out of the system requires tweaking the parameters for the particular use case. These are documented below.

> **Tip**
>
> Tooltips will often mention if a feature has a significant performance cost.

41.1 Quality Parameters

These are available for tweaking out of the box and should be explored on every project:

- See parameters under *Water Renderer* → *Level of Detail* which directly control how much detail is in the water, and therefore the work required to update and render it. These are the primary quality settings from a performance point of view.
- Adjust resolution of simulations on a per-simulation basis.
- Persistent simulations can have their frequency reduced to improve performance.

41.2 Features

- The water shader has accrued a number of features, and has become a reasonably heavy shader. Where possible these are on toggles which can be disabled to will help the rendering cost.
- Disable any unused simulations under *Water Renderer* → *Simulations* will help.
- By default, there are several extra passes to accommodate the *Collision Layers* feature. If this feature is not needed, performance can be improved by disabling it.
- Rendering features under *Water Renderer* → *Rendering* like Motion Vectors, Write Color and Write Depth can be expensive (especially for BIRP).

41.3 Mobile Performance

Mobile is not the primary target for Crest, but the following are some hints on getting better performance:

- Crest can be draw call heavy which mobile platforms can be sensitive to. Together, reducing the *Water Renderer* → *Level of Detail* → *Levels* and increasing the *Water Renderer* → *Level of Detail* → *Scale* minimum can significantly reduce draw calls.

- For demanding platforms that use tile rendering (like the Meta Quest), consider setting the water to opaque, as it will net a significant performance gain. Transparency does not add a lot of value to open ocean scenes.
- Please see the subheadings below for alternative rendering modes for mobile.

41.3.1 Simple Transparency

Preview

This feature is in preview and may be subject to changes or needs refinement.

The water surface shader has a simple transparency mode which does not require sampling from the Opaque Texture or Depth Texture. This can noticeably improve performance for tile-based rendering GPUs like the Meta Quest. This option is available under *Crest/Water* → *Surface Inputs* → *Simple Transparency*.

This feature requires the *Depth Simulation*.

Example

Enable Simple Transparency in the Main sample to see it functioning, as this scene has the Depth Simulation set up with a Depth Probe.

Tip

To take full advantage of the performance this feature can bring: ensure the opaque and depth texture are not being generated. Other effects can still force them to generate. This includes, currently, our own underwater effect, but may change in the future.

41.3.2 Quad Mesh Mode

Preview

This feature is in preview and may be subject to changes or needs refinement.

Renders the water surface using a quad instead of numerous chunks. This can save on draw calls and triangles at the cost of losing displacement. Waves can still contribute to the look of the water with normals and refraction.

CHAPTER

FORTYTWO

RENDERING NOTES

42.1 Transparency

By default Crest is rendered in a typical way for water shaders: in the transparent pass and refracts the scene. The refraction is implemented by sampling the camera's color texture which has opaque surfaces only. It writes to the depth buffer during rendering to ensure overlapping waves are sorted correctly to the camera. The rendering of other transparent objects depends on the case, see headings below. Knowledge of render pipeline features, rendering order and shaders is required to solving incompatibilities.

42.1.1 Transparent Object In Front Of Water Surface

Normal transparent shaders should blend correctly in front of the water surface. However this will not work correctly for refractive objects. Crest will not be available in the camera's color texture when other refractive objects sample from it, as the camera color texture will only contain opaque surfaces. The end result is Crest not being visible behind the refractive object.

Preview

Please see the in-preview feature, *Injection Point* , for a proper solution to this problem.

42.1.2 Transparent Object Behind The Water Surface

Alpha blend and refractive shaders will not render behind the water surface. Other transparent objects will not be part of the camera's color texture when Crest samples from it. The end result is transparent objects not being visible behind Crest.

On the other hand, alpha test / alpha cutout shaders are effectively opaque from a rendering point of view and may be usable in some scenarios.

42.1.3 Transparent Object Underwater

Obsolete

This advice is only applicable to the *Legacy Underwater*. Please see *Integrate Water Volume (Shader Graph)*.

This is tricky because the underwater effect uses the opaque scene depths in order to render the water fog, which will not include transparent objects.

Rendering the transparent object after the underwater pass (occurs right after the transparent pass) with reduced transparency can often be good enough.

CHAPTER

FORTYTHREE

SYSTEM NOTES

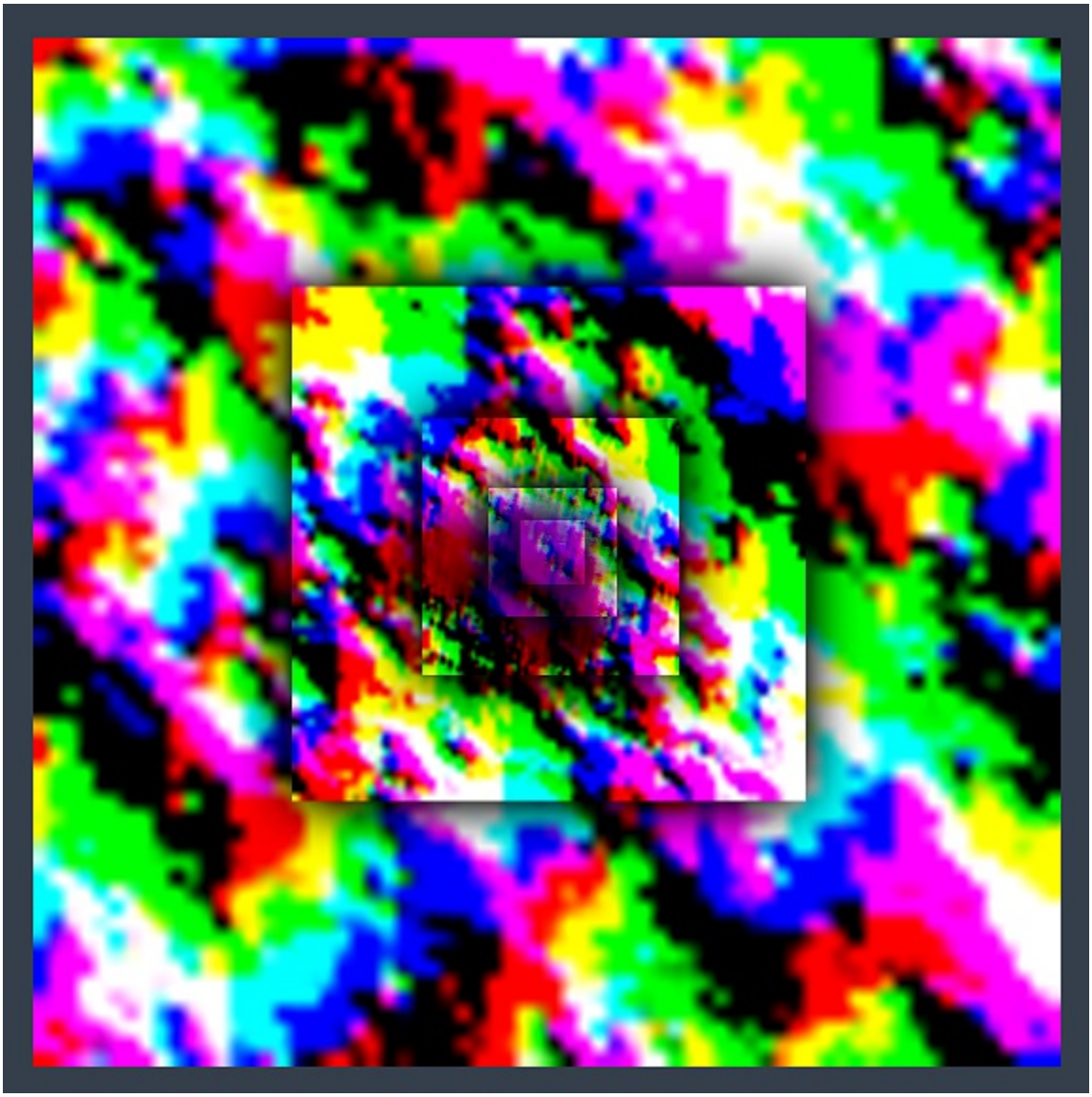
We have published details of the algorithms and approaches we use. See the following publications:

- Crest: *Novel Ocean Rendering Techniques in an Open Source Framework*, Advances in Real-Time Rendering in Games, ACM SIGGRAPH 2017 courses https://advances.realtimerendering.com/s2017/index.html
- *Multi-resolution Ocean Rendering in* Crest Ocean System, Advances in Real-Time Rendering in Games, ACM SIGGRAPH 2019 courses http://advances.realtimerendering.com/s2019/index.htm

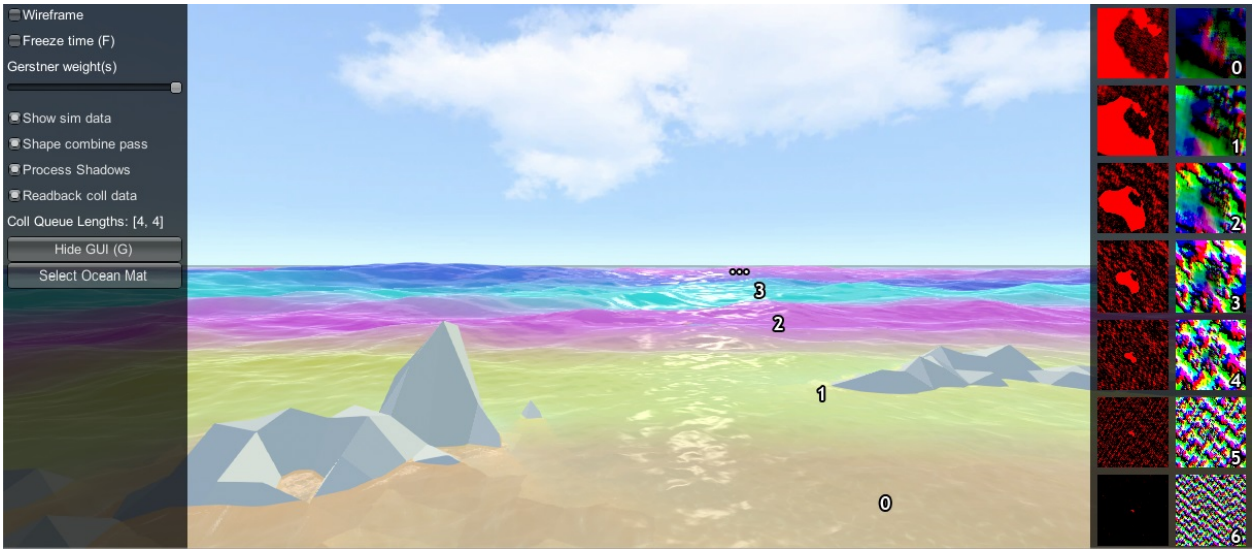
43.1 Core Data Structure

The backbone of Crest is an efficient Level Of Detail (LOD) representation for data that drives the rendering, such as surface shape/displacements, foam values, shadowing data, water depth, and others. This data is stored in a multi-resolution format, namely cascaded textures that are centered at the viewer. This data is generated and then sampled when the water surface geometry is rendered. This is all done on the GPU using a command buffer constructed each frame by *BuildCommandBuffer*.

Let's study one of the LOD data types in more detail. The surface shape is generated by the Animated Waves LOD Data, which maintains a set of *displacement textures* which describe the surface shape. A top down view of these textures laid out in the world looks as follows:

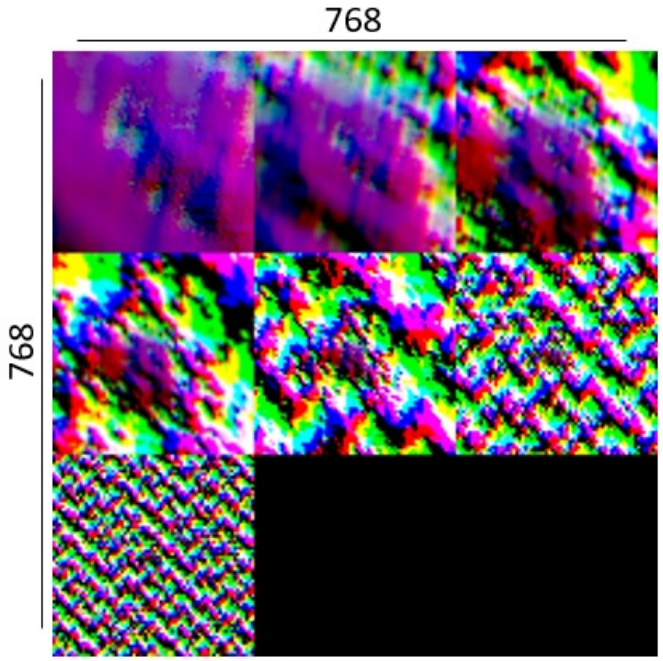


Each LOD is the same resolution (256x256 here), configured on the *Water Renderer* script. In this example the largest LOD covers a large area (4km squared), and the most detail LOD provides plenty of resolution close to the viewer. These textures are visualized in the Debug GUI on the right hand side of the screen:



In the above screenshot the foam data is also visualized (red textures), and the scale of each LOD is clearly visible by looking at the data contained within. In the rendering each LOD is given a false color which shows how the LODs are arranged around the viewer and how they are scaled. Notice also the smooth blend between LODs - LOD data is always interpolated using this blend factor so that there are never pops are hard edges between different resolutions.

In this example the LODs cover a large area in the world with a very modest amount of data. To put this in perspective, the entire LOD chain in this case could be packed into a small texel area:



A final feature of the LOD system is that the LODs change scale with the viewpoint. From an elevated perspective, horizontal range is more important than fine wave details, and the opposite is true when near the surface. The *Water Renderer* has min and max scale settings to set limits on this dynamic range.

When rendering the water, the various LOD data is sampled for each vertex and the vertex is displaced. This means that the data is carried with the waves away from its rest position. For some data like foam this is fine and desirable.

For other data such as the depth to the water floor, this is not a quantity that should move around with the waves and this can currently cause issues, such as shallow water appearing to move with the waves as in #96.

43.2 Implementation Notes

On startup, the *Water Renderer* script initializes the water system and asks the *WaterBuilder* script to build the water surface. As can be seen by inspecting the water at run-time, the surface is composed of concentric rings of geometry tiles. Each ring is given a different power of 2 scale.

At run-time, the water system updates its state in *LateUpdate*, after game state update and animation, etc. *Water Renderer* updates before other scripts and first calculates a position and scale for the water. The water GameObject is placed at sea level under the viewer. A horizontal scale is computed for the water based on the viewer height, as well as a *_viewerAltitudeLevelAlpha* that captures where the camera is between the current scale and the next scale ($\times 2$), and allows a smooth transition between scales to be achieved.

Next any active water data are updated, such as animated waves, simulated foam, simulated waves, etc. The data can be visualized on screen if the *Debug GUI* script from the example content is present in the scene, and if the *Show shape data* on screen toggle is enabled. As one of the water data types, the water shape is generated using an FFT and copied into the animated waves data. Each wave component is rendered into the shape LOD that is appropriate for the wavelength, to prevent over- or under- sampling and maximize efficiency. A final pass combines the shape results from the different FFT components together. Disable the *Shape combine pass* option on the *Debug GUI* to see the shape contents before this pass.

Finally *BuildCommandBuffer* constructs a command buffer to execute the water update on the GPU early in the frame before the graphics queue starts. See the *BuildCommandBuffer* code for the update scheduling and logic.

The water geometry is rendered by Unity as part of the graphics queue, and uses the *Crest/Water* shader. The vertex shader snaps the vertices to grid positions to make them stable. It then computes a *lodAlpha* which starts at 0 for the inside of the LOD and becomes 1 at the outer edge. It is computed from taxicab distance as noted in the course. This value is used to drive the vertex layout transition, to enable a seamless match between the two. The vertex shader then samples any required water data for the current and next LOD scales and uses *lodAlpha* to interpolate them for a smooth transition across displacement textures. Finally, it passes the LOD geometry scale and *lodAlpha* to the water fragment shader.

The fragment shader samples normal and foam maps at 2 different scales, both proportional to the current and next LOD scales, and then interpolates the result using *lodAlpha* for a smooth transition. It combines the normal map with surface normals computed directly from the displacement texture.